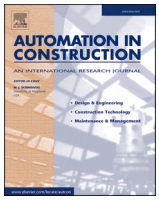
Automation in Construction 191 (2026) 107184

Contents lists available at ScienceDirect

Automation in Construction

journal homepage: www.elsevier.com/locate/autcon



BIM log mining for interactive target value design

Suhyung Jang [a,b,c,*], Ghang Lee [c,d], André Borrmann [a,b], Seokho Hyun [c], Junghun Lee [c], Doyoon Park [e], Seokheon Yun [e]

[a] Georg Nemetschek Institute Artificial Intelligence for the Built World, Technical University of Munich, Munich, Bayern, Germany
[b] Chair of Computing in Civil and Building Engineering, School of Design and Engineering, Technical University of Munich, Munich, Germany
[c] Department of Architecture and Architectural Engineering, Yonsei University, Republic of Korea
[d] Institute for Advanced Study, Technical University of Munich, Germany
[e] Department of Architectural Engineering, Gyeongsang National University, Republic of Korea

ARTICLE INFO

Keywords:
Building information modeling (BIM)
BIM log mining
Target value design (TVD)
Artificial intelligence (AI)-augmented BIM
Shape-constrained regression

ABSTRACT

Even with building information modeling (BIM) adoption, early-stage cost and schedule estimation requires a round-trip to external tools or specialists, decoupling assessment from design iteration. Such delayed assessment breaks the early feedback needed for target value design (TVD), limiting designers' ability to respond early to emerging cost and schedule overruns. This paper proposes an artificial intelligence (AI)-augmented BIM log mining framework that facilitates interactive TVD, focusing on total cost and construction duration. The proposed framework comprises three modules: (1) element-level state logging; (2) cost and duration estimation using shape-constrained machine learning regressors; and (3) in-authoring visualization of estimation trajectories. Validation on 93 projects shows mean absolute percentage error (MAPE) of 41.2% for cost and 21.3% for duration. The proposed t-dependent origin anchoring transformation addresses low-count extrapolation violations, yielding logically consistent estimates for early design states underrepresented during training. State-log-based feature updates keep interactive latency below 100 milliseconds, providing immediate feedback.

1. Introduction

Decisions made during the schematic design phase, such as spatial layouts and building element dimensions, directly affect key performance outcomes of building projects such as cost and schedule. However, traditional cost and schedule estimation workflows postpone such insights until interim or complete design submittals are prepared. For example, conventional cost estimation relies on a set of completed design documents for quantity takeoff, followed by manual unit price assignment using external cost databases [1]. Even with the adoption of building information modeling (BIM), through which quantities can be extracted automatically, cost and schedule estimation remains largely disconnected from ongoing design iterations [2]. This disconnect persists because current practice still relies on document-level extraction workflows and external cost and schedule databases that operate outside the BIM authoring environment. Consequently, designers cannot continuously understand whether ongoing early design decisions are likely to move a project toward or away from its project-level cost and duration targets.

This lag in feedback limits the practical implementation of 'target value design (TVD)'which requires design decisions to be informed by continuously updated target values while the design is still evolving. As illustrated by the McLeamy Curve [3], the ability to influence project outcomes is greatest in early design, whereas the cost of design changes increases substantially as the project progresses. In current separated design-estimation-rework workflows, the cost or schedule overruns are often identified only after an interim design output has been produced, requiring designs to be revised and re-estimated [4]. TVD emphasizes early and frequent updating of cost estimates and estimated values precisely to reduce such delayed correction cycle [5]. Especially, TVD requires designers to understand the cost and schedule implications of design decisions during authoring, rather than only after a separate estimation process. At the schematic design stage, this need concerns timely indicative feedback on project-level total cost and construction duration. However, there are several dilemmas that must be addressed to enable such early and timely cost and schedule estimation.

The first dilemma lies in the timing mismatch. Typically, construction cost and schedule estimation cannot be executed until interim

* Corresponding author at: Georg Nemetschek Institute Artificial Intelligence for the Built World, Technical University of Munich, Munich, Bayern, Germany.
E-mail addresses: suhyung.jang@tum.de (S. Jang), glee@yonsei.ac.kr (G. Lee), andre.borrmann@tum.de (A. Borrmann), tjrgh25@yonsei.ac.kr (S. Hyun), dlwjdgjs98@yonsei.ac.kr (J. Lee), hebelube@gnu.ac.kr (D. Park), gfyun@gnu.ac.kr (S. Yun).

https://doi.org/10.1016/j.autcon.2026.107184
Received 22 February 2026; Received in revised form 23 July 2026; Accepted 27 July 2026

design outputs are available. By that time, any design revision incurs additional time and cost, reflecting required changes after the estimation process becomes more expensive and less efficient. The second dilemma arises from the lack of detailed information in early-stage schematic designs, which hinders accurate cost and schedule estimation [6]. Current practice often relies on static historical data to provide early cost estimates using broad project-scale indicators such as gross floor area (GFA), or number of beds or seats. While these heuristics offer quick approximations, their accuracy is limited by nature. According to the American Association of Cost Engineering (AACE), estimates in this phase may deviate by −30% to +50% [7]. Moreover, such methods are only applicable when a sufficiently similar project exists in the estimator's legacy database [8]. As a result, design firms with limited experience often outsource cost estimation to specialized consultants, further delaying feedback. Analogous issues arise for schedule estimation, where early assessments are typically based on coarse productivity assumptions and high-level analogies to past projects rather than project-specific model information.

A further practical limitation is that during ongoing BIM authoring, there is no continuously available data stream that can be used directly for cost and schedule estimation. Although extensive studies have proposed methods utilizing artificial intelligence (AI)-based cost and schedule estimation, most approaches assume that features extracted from static BIM model at a certain state. However, constantly retrieving BIM properties as they are being authored causes a lack in supporting immediate interactive responses. For example, parsing a BIM model with less than 20 building elements, each containing multiple property fields, entails processing times exceeding 100 milliseconds, which is the real-time support threshold for interactive latency in exploratory visual analysis [9]. When the model includes more than dozens building elements, processing time surpasses 1000 milliseconds (one second), which is considered a cognitive operation on a standard notebook computer (detailed settings are provided in Section 5.3). In practical cases with several thousand building elements, processing time exceeds 10,000 milliseconds (ten seconds), which is perceived as a unit task rather than immediate feedback. Although such delay may not be problematic when extraction and estimation are conducted after a specific design step, as in conventional workflows, it becomes a significant issue if the same delay occurs for every design decision required for interactive TVD.

Even if the interactive latency issue were mitigated, there remains a mismatch between how the estimators are trained and how they are deployed during early design stage authoring. In usual cases, training data for cost and schedule estimators are derived from late-stage BIM models of completed projects, where quantity-type features (e.g., counts, areas, volumes) are fully populated. In contrast, early design authoring states are characterized by low-count inputs, requiring the estimator to extrapolate far outside the training distribution. This training–deployment discrepancy leads to extrapolation violations, where estimates for unseen distributions fail to follow logical cost and schedule trends. Therefore, early-stage cost and schedule estimation requires estimation models and input representations that are explicitly robust to low-count states and out-of-distribution extrapolation.

These practical barriers motivate the need for an alternative data source that is available continuously during authoring, while being lightweight for incremental updating. In this context, BIM logs are promising candidates because they record a time-sequential stream of authoring actions during BIM tool use. A growing body of research has examined BIM log data to discover user patterns, compare as-happened and as-planned BIM processes, and predict BIM commands [10]. However, BIM logs in commercial BIM authoring tools were designed for system maintenance, not to support understand design-oriented process [10,11]. Consequently, such logs often omit the state changes needed to interpret design revisions made at the BIM element-level [12]. Even though efforts have been made to enhance these logs through custom loggers, these adaptations still are not able to explicitly capture geometric and attributive details of building elements to understand the results of the decision-making involved in the BIM authorship [13,14]. Accordingly, leveraging BIM logs as a reliable data source for cost and schedule estimation requires an element-level, state-aware logging mechanism that records semantic and geometric changes with interactive latency.

To address these limitations, this paper introduces a BIM log mining framework that enables interactive project-level cost and construction duration feedback during early design stage BIM authoring, while addressing extrapolation violations in unseen input distribution. The framework consists of three core modules: (1) an event logging module that captures element-level state changes as modeling progresses; (2) a target value estimation module that incrementally updates input features in response to logged changes to estimate project targets such as cost and duration; and (3) a trend visualization module that displays performance trajectories during authoring. In combination, these modules provide immediate feedback on estimated cost and duration from early design stage BIM models to support designer's understanding the implication of design changes as they happen. Rather than repeatedly extracting estimation features from the evolving BIM model, the proposed method uses element-level BIM logs as an incrementally updated data source for estimation during ongoing authoring. In addition, this paper introduces a shape-constrained AI deployment strategy named the *t*-dependent origin anchoring transformation, where t denotes the authoring state index. This transformation allows estimators trained on finalized project states to be applied to early design stage, imposing stronger shape constraints on low-count input configurations that were absent or rare in training distribution. The framework was implemented as an Autodesk Revit C# add-on supported by an Open Neural Network Exchange (ONNX)–based cost and duration estimators. Validation experiments on 93 projects were designed as an ablation study of multiple gradient-boosted regressors with and without monotonic constraints and *t*-dependent origin anchoring transformation, and the proposed interactive cost and duration feedback framework was evaluated in terms of accuracy, low-count extrapolation behavior, and end-to-end interaction latency.

This study makes three main contributions. First, it proposes a framework that can provide immediate cost and duration feedback during early design stage BIM authoring. This allows TVD to be supported within the design process rather than through repeated estimation steps carried out separately from design progress. Second, it presents a BIM logger and corresponding log schema for capturing enriched element-level BIM state changes. It also shows that these logs can be used as a practical input source for machine-learning-based estimation during BIM authoring, extending BIM log mining beyond post-hoc process analysis. Because the proposed log schema is extensible to additional geometric and semantic properties, it also provides a foundation for future estimation modules incorporating additional input features and addressing performance targets beyond the total construction cost and construction duration evaluated in this study. Third, it proposes the t-dependent origin anchoring transformation, which addresses extrapolation violations in early BIM states that are not represented in the training data. Together, these contributions support more continuous cost- and duration-aware decision-making during early-stage BIM authoring.

The remainder of this paper is organized as follows: Section 2 reviews previous work in BIM-based cost and duration estimation and BIM log mining to identify research gaps. Section 3 describes the architecture and implementation of the proposed framework. Section 4 presents experimental results evaluating the accuracy and responsiveness of the system. Section 5 discusses potential applications and future research directions. Section 6 concludes the study.

2. Background

Early design stage construction cost and duration estimation, relies on project-scale-based indicators such as GFA or number of functional

Table 1
Summary of previous BIM-based cost and schedule estimation approaches.

Approach	Target	Authors	Year	Proposed cost estimation method	References
Mapping-based estimation	Cost	Cheung et al.	2012	Link Royal Institution of Chartered Surveyor (RICS) New Rules of Measurement (NRM) with geometric parameters extracted from BIM models.	[1]
		Ma et al.	2013	Link Chinese national mandatory specification (GB0500–2008) with BOQ parsed from BIM models.	[16]
		Plebankiewicz et al.	2015	Link experienced cost estimators' unit cost presets with geometric properties extracted from IFC elements.	[17]
		Fazeli et al.	2019	Link Iranian cost estimation standard (FehrestBaha) with BOQ parsed from BIM models.	[18]
	Schedule	Kim et al.	2013	Generate schedule activities and durations from IFC attributes and production rates.	[19]
		Moon et al.	2015	Adjust overlaps in BIM-linked schedules via risk-based optimization.	[20]
		Wang and Rezazadeh Azar	2019	Formwork packages and draft schedules from BIM quantities and rule-based relationships.	[21]
Heuristic estimation	Cost	Park and Yun	2023	Predict total cost from project attributes and element quantities using a multi-layer perceptron (MLP).	[22]
		Lee and Yun	2024	Improve machine-learning-based cost datasets using imputation and outlier removal.	[23]
		Zhang and Mo	2024	Predict and optimize costs from BIM quantities and project attributes an Elman neural network (ENN).	[24]
	Schedule	Sigalov and König	2017	Identify recurring process patterns in BIM-based schedules for template reuse.	[25]
		Lagos et al.	2024	Predict schedule performance from Last Planner and schedule control indicators.	[26]
		Wefki et al.	2024	Generate and optimize BIM-based schedules using GA and 5D simulation.	[27]

units [15]. While this approach can provide early financial insights, its accuracy is limited, with an expected accuracy range between −30% and + 50% [7]. To improve accuracy, contractors and owners increasingly adopt BIM component-level estimations, such as aggregated areas or volumes of building elements, for more precise predictions [6]. However, despite the growing reliance on BIM models, current systems face inherent limitations in supporting real-time cost and schedule estimation [10], particularly during early design stages.

2.1. BIM-based cost and schedule estimation

Cost and schedule estimation are the most widely studied and practically relevant applications of BIM. Previous approaches to BIM-based early design stage cost and schedule estimation can be classified into two categories (Table 1): (1) mapping-based estimation, which associates a BIM-derived bill of quantities (BOQ) with a unit price or work-breakdown structure (WBS) database, and (2) heuristic estimation, which predicts aggregated performance based on dimensional variables. The former provides estimates through explicit mappings to construction-related information, whereas the latter supports approximate assessment without requiring fully resolved cost items or construction processes.

Linking a BOQ with a unit price database requires either extracting geometric parameters or parsing semantic attributes such as Industry Foundation Classes (IFC) properties or material information within the authoring software. Cheung et al. [1] proposed a method to automatically parse geometric features at various design stages of a building project, generating evolving BOQs directly from 3D building models created in SketchUp. Their approach leverages floor-area-level, functional-unit-level, and element-level quantity takeoffs, linking these quantities with the cost model prescribed by the Royal Institution of Chartered Surveyors (RICS) New Rules of Measurement (NRM). Similarly, Plebankiewicz et al. [17] proposed a method to extract IFC geometric properties and applied predefined unit costs from experienced estimators. Ma et al. [16] and Fazeli et al. [18] proposed approaches that integrate BOQs structured from BIM elements' geometric attributes with regional cost estimating standards such as China's National Mandatory Specification for Tendering of Building Projects Cost Estimation (GB0500–2008) and the Iranian Cost Estimation Standard (Fehrest-Baha). Several commercially available software solutions, such as RIB CostX, Autodesk Assemble, BIMestiMate [17], DProfiler [28] and Vico Cost Planner [29], employ similar approaches to associating BOQs with unit price databases.

Automated schedule estimation studies also apply the same mapping-based approach to generalized construction activities. Kim et al. [19] extracted spatial, geometric, and material-layer information from IFC-based BIM models, generated activities, computed durations from production rates, and established sequencing rules to obtain a detailed schedule. Moon et al. [20] took a BIM-linked schedule and optimized risky activity overlaps under precedence and resource constraints. Wang and Rezazadeh Azar [21] extracted dimensions, quantities and spatial information, grouped tasks into work packages, computed durations from production rates, and used rule- and case-based reasoning to derive relationships and produce draft schedules. Across these works, estimation of building projects using BIM models is tightly coupled to specific metadata mappings from elements to unit prices, work packages, and activities.

A second group of methods relaxes the requirement for fully coded metadata and instead predicts aggregated performance from dimensional and indicator-level features. Rough estimation based on variables such as total area, number of beds, or other functional units has traditionally been used to estimate project-level performance from historical data. More recently, developments in artificial intelligence have extended this heuristic approach by enabling predictive models to learn relationships between dimensional variables and historical performance outcomes. Park and Yun [22] used a multi-layer perceptron (MLP) to estimate total project cost from general project attributes (e.g., site area, building area, landscape area, number of floors, parking spaces) and quantities of various element types (e.g., rooms, walls, floors, columns, stairs, openings). Lee and Yun [23] improved robustness through missing-value imputation and outlier removal. Zhang and Mo [24] combined BIM-derived quantities with project attributes such as labor and equipment and used an Elman neural network (ENN) to predict and optimize construction costs. Here, the model operates on a compact feature vector of areas, volumes, counts and project descriptors instead of element-level code mappings.

Previous studies in schedule estimation using heuristic prediction shows a similar shift toward aggregated indicators or higher-level schedule information. Lagos et al. [26] proposed a method to predict schedule using last planner system (LPS) indicators and traditional schedule indices, providing early warnings of schedule deviations without regenerating the full activity network. Sigalov and König [25] detected recurring process patterns in BIM-based schedules, enabling template reuse and higher-level reasoning on schedule structure. Wefki et al. [27] combined BIM-derived quantities with genetic algorithms and 5D simulation to generate and optimize schedules. In all these cases, cost

and schedule are inferred from dimensional, structural, or management indicators, rather than from direct one-to-one mappings between individual BIM elements and cost or schedule units.

Accordingly, mapping-based approaches are suitable for deriving detailed cost breakdowns and activity-level schedules when sufficiently defined construction information is available, whereas heuristic approaches are more applicable to approximate project-level cost and duration estimation when such detailed information is not yet available. However, the applicability of both approaches to ongoing estimation during early design depends on the information available in intermediate BIM states, as discussed in the following section.

2.2. Limited information basis for estimation during early design

Unlike a detailed BIM model at the design development or construction documentation stage, an early schematic design model focuses on exploring layouts, boundaries, openings, circulation, and overall project scale rather than detailed building element properties such as materials, exact dimensions, and component assemblies. This detailed information is significant in understanding the required construction works and the amount of resources, including materials, labor, and equipment. However, much of this information is either unavailable or likely to change during early design.

Nevertheless, not all information represented in an early BIM model is equally unstable. While material specifications, component layers, and detailed assemblies are progressively determined and frequently revised, the primitive geometric dimensions that define the developing building can remain relatively stable. For example, as a wall object is developed, the function, material, and thickness of each layer can be specified and revised according to the selected construction solution [30]. Accordingly, the quantities associated with the completed wall assembly can substantially change. However, the primitive geometry of the wall, such as its vertical profile surface, can remain a comparatively stable indicator of the scale of the developing design.

This distinction is illustrated in Fig. 1 through the representation of a wall and its openings at different levels of development. In the low-detail representation, consistent with level of development (LOD) 200, the wall and openings are represented as approximate single elements. In the detailed representation, consistent with LOD 400, the wall is represented as a layered assembly, while the openings are represented through detailed components with resolved dimensional and functional information. A similar principle applies to planar building elements such as walls, floors, windows, and doors. For stairs, an early representation can indicate approximate location and quantity required for layout and circulation planning, whereas more detailed representations include resolved information regarding landings, railings, rises, treads, and construction-related components. Thus, early BIM states can provide relatively stable geometric indicators of project scale, while lacking the detailed construction information required for construction-resolved estimation.

Under these conditions, the purpose of estimation in early-stage Target Value Design (TVD) differs from that of detailed cost planning or construction scheduling. TVD requires design decisions to be evaluated against project targets while the design is still evolving. Therefore, even when detailed construction information is unavailable, heuristic estimation based on the approximate scale of the developing project can support ongoing design decisions by estimating total construction cost and total duration and indicating the risk of exceeding cost or schedule targets. In this context, the objective is not to generate a detailed discipline-specific cost breakdown or an executable activity-level schedule, but to provide project-level feedback for target-compliant design decision-making.

However, supporting such feedback during ongoing BIM authoring requires not only information that is available in early design, but also a

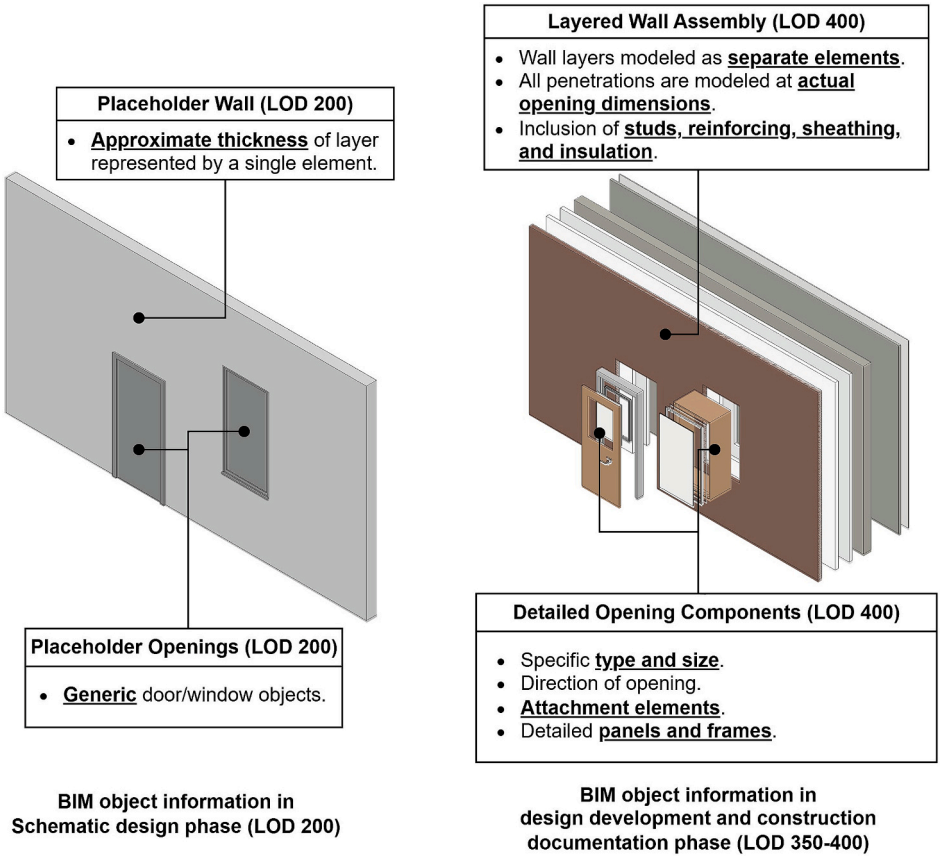


Fig. 1. Comparison of information granularity between LOD 200 (left), and LOD 400 (right) BIM elements (revised from Jang and Lee, 2024 [31]).

mechanism for continuously capturing evolving design information and updating estimation inputs as authoring progresses.

2.3. BIM log mining

BIM logs, records of events generated during BIM authoring [10], offer unique potential to capture as-happening design process. Over the past decade, a growing number of studies have explored BIM log mining to discover, analyze, and improve design process. To better understand these efforts, it is useful to align them with established categories of process mining [49]: (1) process discovery, (2) conformance checking, and (3) process enhancement, as illustrated in Fig. 2.

2.3.1. Process discovery

The most explored category is process discovery. Process discovery aims to reconstruct and analyze as-happened BIM processes from BIM logs. In process discovery, event traces are used to derive descriptive process models and to characterize how modeling is actually performed, including frequent action patterns, typical paths, and rework behaviors. Reported use cases include identifying design authoring characteristics [32,33], analyzing collaboration patterns [34,35], analyzing design task-level productivity [36], discovering authoring processes [37], and discovering construction processes [38,39].

2.3.2. Conformance checking

Conformance checking focuses on whether an as-happened BIM authoring or construction process conforms to as-planned procedure. The as-planned procedure can be represented as a reference workflow, while BIM logs provide time-sequential traces of executed actions. Conformance checking involves comparing BIM logs with an expected procedure to detect deviations such as missing required steps, unexpected extra actions, incorrect ordering, excessive rework loops, or abnormal delays. Use cases include assessing whether as-happened design or construction records conform to as-planned design processes [37,39].

2.3.3. Process enhancement

Process enhancement aims to improve future modeling actions by using mined results to support prediction, recommendation, or optimization of subsequent steps. Unlike discovery and conformance checking, which are primarily diagnostic, enhancement is related to action-oriented improvements. Examples include predicting authoring commands [40,41] and forecasting construction progress [42].

Compared to manufacturing process mining which exhibits mature implications of process enhancement, BIM log mining remains largely retrospective, mainly focused on process discovery. Attempts for process enhancement in BIM log mining are limited to authoring command predictions. A key challenge is the limited granularity of metadata in BIM logs. Unlike manufacturing environment where production lines are fixed, all building projects are unique and their design workflows vary. Hence, understanding such process requires element-specific interpretations rather than event-level comprehension [10]. However, current native BIM logs are designed for system monitoring and hence omit element-level details [10]. While there have been efforts to address such data scarcity by developing custom loggers (Table 2), a core limitation persists. The loggers developed by Gao et al. [43] and Jang et al. [44] can capture instances affected by design commands but fail to record the evolving geometric and semantic features of elements in three-dimensional modeling environments (e.g., Rhino). The BIM logger proposed by Kouhestani and Nik-Bakht [37], later enhanced by Pan and Zhang [39], captures attributive features but does not support geometric feature logging. Moreover, their loggers rely on comparison between consecutive versions of IFC models, therefore, cannot be applied to support real-time decision-making. The logger developed by Yarmohammadi and Castro-Lacouture [12] records instance identifiers (IDs) and has the potential to log semantic properties using their proposed method. However, it represents element geometry only through simplified bounding boxes. Although these bounding boxes approximate BIM element geometries, they can substantially distort actual shapes and thus mislead building design interpretation [6].

Table 2
Comparison of real-time support and contents in BIM logs.

BIM logger	Real-time support	Inclusion of content		
		Instance identifier (ID)	Geometric properties	Semantic properties
Native BIM logs [10]	✓	–	–	–
Gao et al. [43] and Jang et al. [44]	✓	✓	–	–
IFC logger [37,39]	–	✓	- (Possible)	✓
Yarmohammadi and Castro-Lacouture [12]	✓	✓	✓ (Bounding box)	- (Possible)
Enhanced BIM logger (proposed)	✓	✓	✓	✓

Beyond content limitations, the rigid formats of BIM logs (i.e., structured text or tables) restrict flexibility. Elements representations and associated properties vary across projects and use cases, but fixed schemas do not accommodate this hierarchical and content diversity. Together, semantic insufficiency and structural inflexibility limit the use of BIM logs for in-use process enhancement and dynamic analysis during BIM authoring. As a result, BIM log mining is often confined to post-hoc analysis, reducing its value for time-sensitive decisions that require reliable estimations during early-stage authoring.

2.4. Research gaps and objectives

This study reframes BIM log mining as a dynamic decision-support mechanism embedded in BIM authoring, rather than serving as a passive post-hoc analysis. Building on this reframing, the paper presents an interactive cost and duration feedback framework that supports TVD workflow by delivering estimates of total construction cost and total duration to designers while they are authoring BIM models. By doing so, it aims to provide timely indicative feedback that allows designers to understand whether an evolving design is likely to remain within predefined cost and duration targets. Fig. 3 contrasts the conventional design-estimate-revise TVD workflow with the proposed interactive feedback-based TVD workflow.

In the conventional design-estimate-revise TVD workflow (Fig. 3A), target compliance is assessed only after interim or final design result is produced. Design outputs are exported to external experts or tools, where quantities or dimensional features are extracted from the BIM model and used for cost and schedule estimation. Mapping-based

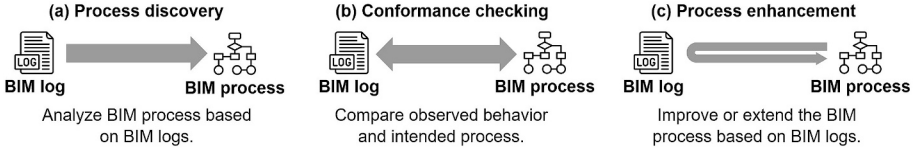


Fig. 2. Three basic types of BIM log mining.

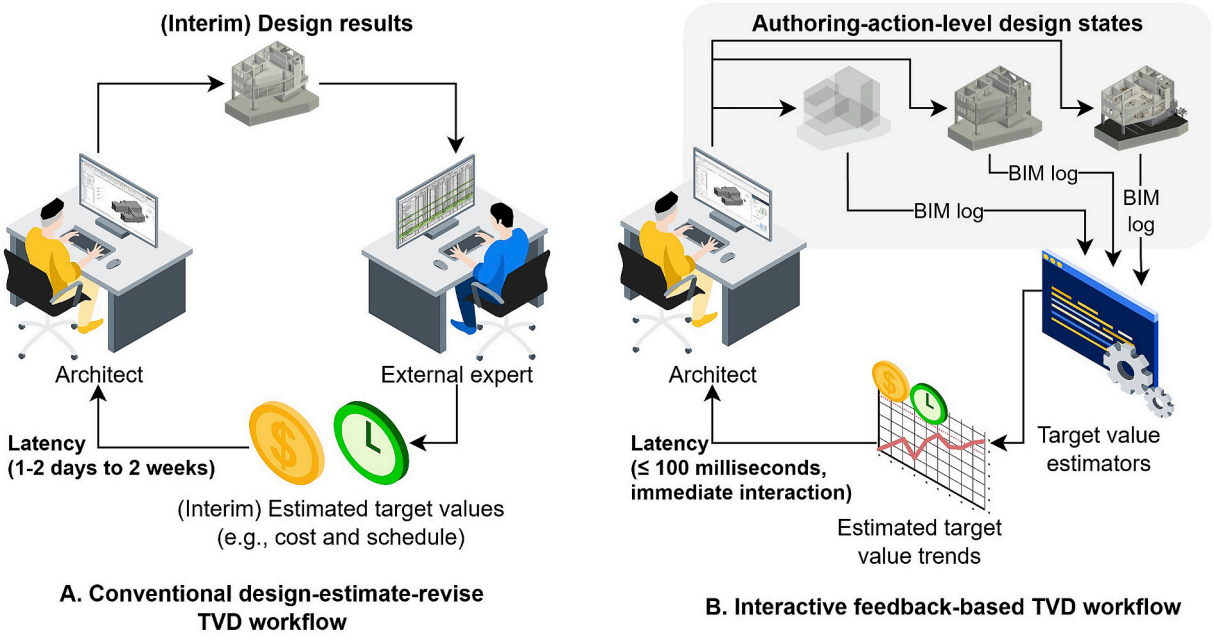


Fig. 3. Comparison of (A) conventional design-estimate-revise workflow and (B) interactive feedback-based TVD workflow.

approaches may require full-model quantity takeoff together with rule- or database-based mappings to unit prices, work packages, productivity rates, and activity relationships. Heuristic approaches may reduce the need for such detailed mappings, but they still require the extraction of dimensional or quantity-based features from the current BIM state. In either case, cost and duration feedback is obtained after a design state has been produced and is therefore decoupled from the moment at which individual design decisions are made. This leads to repeated design-estimate-revise cycles and limits the ability of designers to evaluate target compliance continuously during authoring.

In contrast, the proposed interactive feedback-based TVD workflow (Fig. 3B) enables designers to continuously understand the estimated consequences of their authoring actions as design evolves. Along with each authoring action (e.g., element creation or modification), estimation-related input features are incrementally updated from BIM logs. The corresponding estimates of total construction cost and total duration are then visualized together with their trends and the target thresholds. In schematic design, such feedback can support comparisons among alternative massing, layout, or element-level decisions as they are being authored, allowing designers to judge whether a design change is likely to move the project closer to or farther from its predefined targets. Rather than waiting for a separate estimation cycle, designers can revise the current design direction based on immediate feedback regarding the estimated consequences of their actions.

However, two technical limitations remain when it is applied during ongoing BIM authoring. First, conventional BIM-based estimation workflows rely on repeated extraction of required input features from successive BIM model states. As project size and the number of required features increase, this repeated full-model extraction limits the provision of immediate feedback. Although BIM logs offer the potential to update estimation-related features incrementally, existing BIM logging approaches do not provide real-time element-level geometric and semantic information.

Second, the reliability of heuristic estimation during ongoing early design remains unresolved. Most available training data in the construction industry, such as design documents paired with cost or duration outcomes, are obtained from completed projects or sufficiently developed design states. In contrast, estimation during ongoing early design requires predictions from intermediate BIM states containing substantially smaller quantity-type features, such as counts, areas, and volumes. This mismatch between training and deployment input feature distributions can result in extrapolation violations [45], including implausible estimates at low-information states or non-monotonic changes in estimated cost or duration as the modeled project scope increases. These extrapolation violations are further examined in Section 3.2.2.

Accordingly, despite recent academic and commercial efforts toward BIM-based cost and schedule estimation, a research gap remains in supporting reliable and immediate project-level cost and schedule estimation during ongoing early-stage BIM authoring. Existing mapping-based approaches require construction-related information that is not sufficiently defined in early design. Existing heuristic approaches are more suitable for approximate target monitoring, but they are limited by repeated feature extraction and extrapolation risks when applied to intermediate design states. Meanwhile, existing BIM log mining approaches do not capture the evolving geometric and semantic contents required to incrementally update estimation features in real time.

To address these existing gaps, this study aims to achieve following research objectives:

(1) To propose an extensible BIM log data structure and recording system that captures element-level geometric and semantic properties;
(2) To develop a framework that leverages the enriched BIM logs to deliver accurate and computationally efficient total cost and construction duration estimations during early design stage authoring; and
(3) To develop a method for addressing extrapolation violations that arise when estimators trained on finalized project datasets are used for in-authoring estimation.

Although the effectiveness of TVD workflow is well-recognized at the management level and early, its successful implementation requires early and frequent total cost and construction duration analysis. However, techniques for providing such feedback during early design stage BIM authoring remains limited. To address this gap, this study presents a framework that uses enriched BIM logs to provide interactive feedback on the expected cost and schedule consequences of authoring actions as the BIM model evolves.

3. Interactive cost and schedule feedback framework

The proposed framework consists of: (1) *event logging*, (2) *target value estimation*, and (3) *trend visualization* modules (Fig. 4). Among these modules, *event logging* and *target value estimation* modules run on the back-end of BIM authoring tools: the *event logging* module records detailed event occurrence (e.g., addition, deletion, and modification of building elements). The *target value estimation* module, utilizing pre-trained AI estimators, receives event logs, updates the input features for the estimation, and estimates target value (e.g., total cost and construction duration) to return the results. The estimation results are then updated in the front-end *trend visualization* module. The *trend visualization* module delivers interactive feedback by displaying estimated target value trends prompted by the design actions made by the designer.

3.1. Event logging module

The event logging module records user-initiated authoring actions and corresponding BIM element changes during design authoring. It builds on instrumented event arguments in the BIM authoring system that audit user design actions, including creating, saving, and closing a project or session, as well as element-level creation, revision, and deletion [12,44]. Because event logs store only these deltas, they remain memory-efficient while still capturing a diverse range of BIM properties. In addition to the event logs written during these design actions, we introduce a state log that stores the current snapshot of the BIM model at a given instant by listing all elements with their latest semantic and geometric properties. For each authoring action, the module writes an event log and updates the state log. For creation and deletion events, it updates the state log by adding or removing the corresponding element exactly as indicated in the event log. For revision events, it updates the element in the state log, compares the element's record before and after the update, and records in the event log only the incremental delta in geometric or semantic attributes. This approach prevents repeated model parsing to obtain the current BIM state. It maintains a live state log that can be passed directly to the target value estimation module. Fig. 5 illustrates event and state logs for a scenario with two wall elements with IDs 220,624 and 250,301, which are created in sequence, followed by a property change (i.e., change of instance class) for element 220,624 and its subsequent deletion.

The data schema and contents extend the authors prior efforts to develop a BIM logging system which enables reproducing the original BIM authoring based on BIM logs [46]. The previous logging system stores BIM logs in comma-separated value (CSV) format, a fixed tabular data format. However, these CSV data in fixed tabular format depict limitations. First, it necessitates extensive null values because properties apply selectively by element class. Both windows and walls may have a height property, but sill height applies only to windows. Recording such class-dependent properties in a fixed table forces nulls for non-applicable fields, inflating file size as the cross-class property set grows. This extensively increases the amount of data written (i.e., file sizes) as recorded properties across the classes increase. Second, strict column schemas hinder adding custom properties needed for estimation. Even minor scheme changes demand substantial reconfiguration of the logging stack. These structural rigidity and lack of flexibility make fixed tables a poor fit for capturing diverse element features during BIM authoring.

To address these limitations, this study adopts structured data serialization using java script object notation (JSON) as the primary format, while remaining compatible with extensible markup language (XML) and YAML Ain't Markup language (YAML). In these structured data serialization formats, there are two fundamental entity types, arrays and objects. These two concepts align the serialization approach with the requirements of BIM authoring event logging. Arrays can encode the sequential nature of authoring sessions and events, as multiple sessions can occur in a project, and multiple events can occur within a session. Objects with multiple key–value pairs as properties can flexibly handle the heterogeneous building elements authored in BIM models. This object schema can also represent multi-level hierarchical metadata that conforms to the data structure of BIM systems, which are typically built on object-oriented programming concepts [31].

The proposed BIM log schema defines four independent hierarchies (i.e., project, session, event, and element), which together represent the authoring context, the sequence of interactions, and the affected

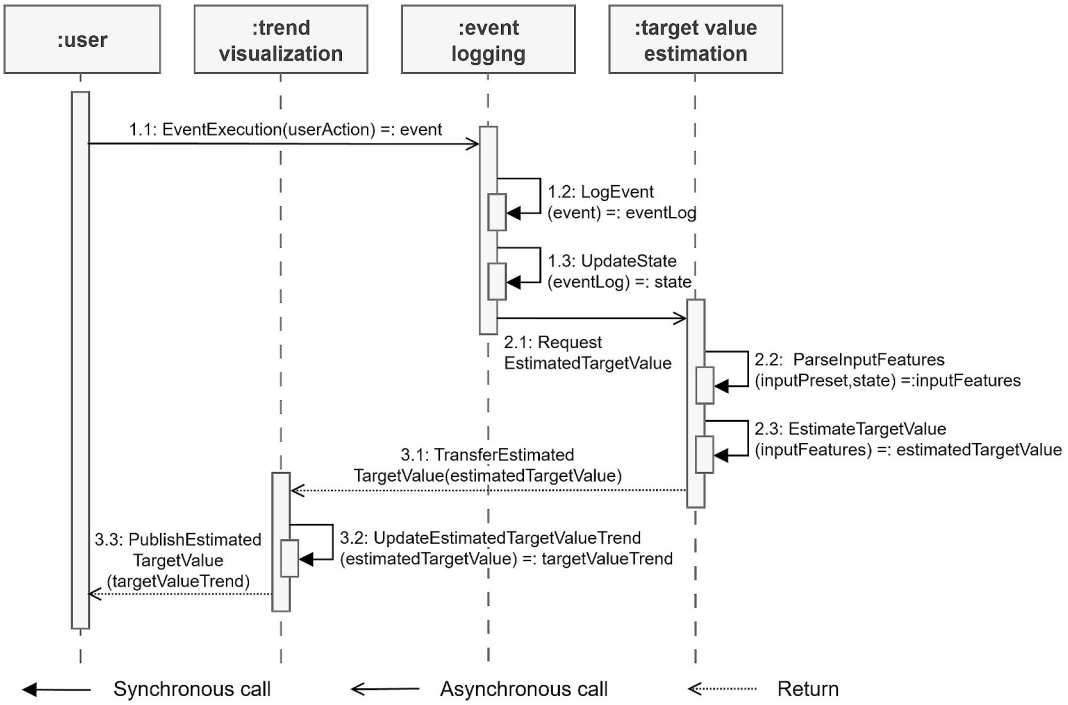


Fig. 4. UML sequence diagram of simultaneous cost-estimation method in BIM authoring environment.

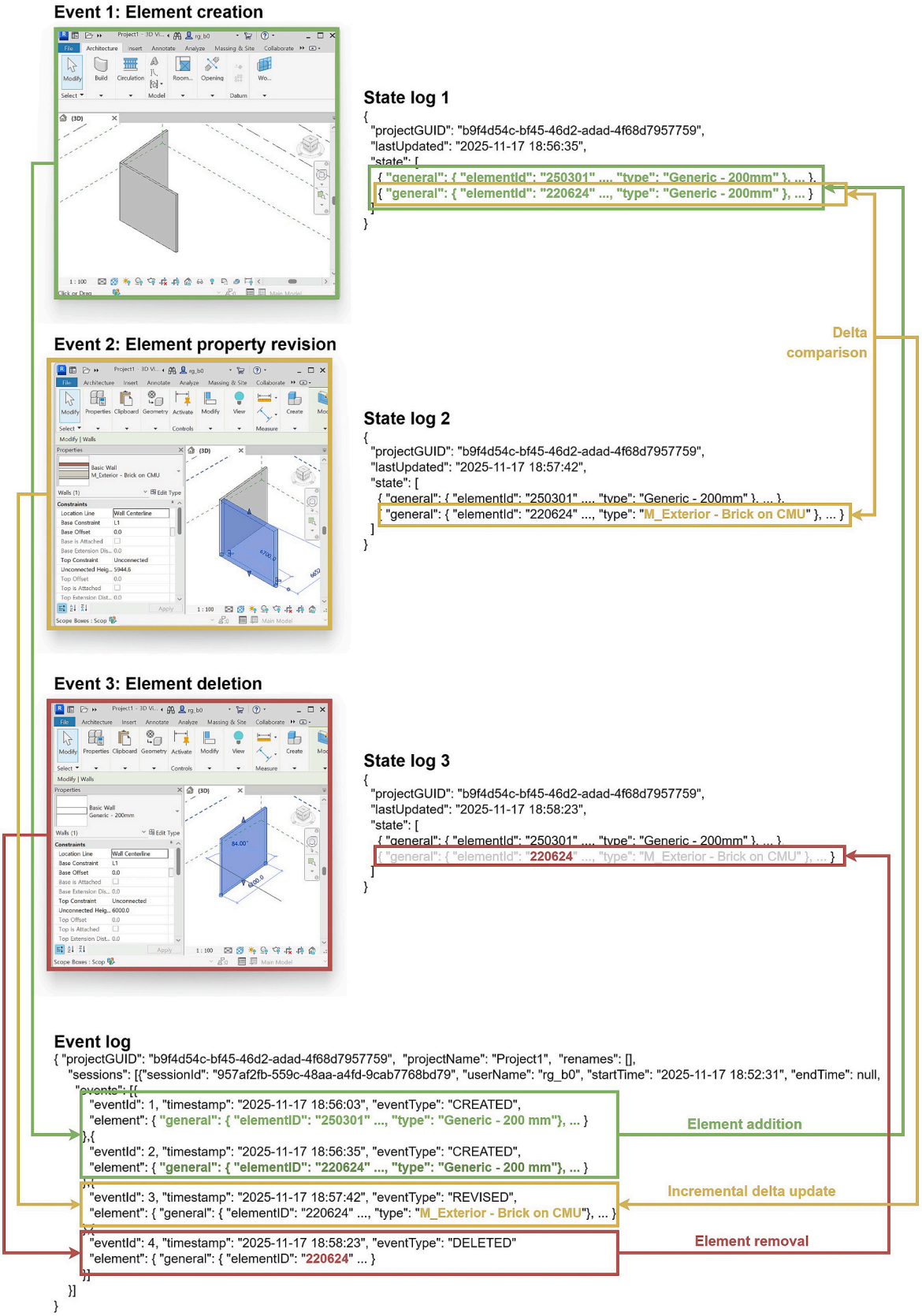


Fig. 5. Event logging mechanism between event occurrence, event log, and state log.

building objects. For formal representation of the scheme and contents, JSON schemas for logging BIM authoring are presented, which were designed to record BIM authoring events in Autodesk Revit 2025 as an example. The schemas and executable files for installing the logger in Autodesk Revit 2025 are available at https://suhyungJang.github.io/revit-logger. The repository defines the authoring context (i.e., project, session, event), the affected element and their parameters (i.e., category-, family-, type-, and instance-specific parameters), as well as the geometric placeholders used during instantiation of the elements. The project is a top-level artifact. It is assigned a persistent project globally unique identifier (GUID) at creation and retains this ID across "Save As" or "Save" operations and across sessions on the same project BIM model. The project object contains a project name and an optional rename history to track name changes over time, as well as an array of sessions (Listing 1). A session is appended each time the authoring tool is opened for that project; it includes a session identification (ID), a user name, a start time recorded at session start, an end time recorded upon closure, and an ordered array of events (Listing 2).

Within a session, an event records element-level authoring actions such as creation, revision, or deletion. Although commands may be grouped at the user interface level, the focus here is on element-centric changes relevant to estimation rather than interaction modalities. Element-level changes between the previous and current model are interpreted using the data-change taxonomy of Lee et al. [47], namely *created*, *deleted*, *preserved*, and *revised* (Table 3). The logs record only mutating events (i.e., *created*, *deleted*, and *revised*) because *preserved* denotes no change to an element's state. Each event is stored with its type, an event ID, a timestamp, and the affected element (Listing 3).

An element denotes any modeled object in the BIM environment, including spatial context objects such as levels and rooms, reference objects such as grids, and building components such as walls, floors, roofs, ceilings, windows, doors, columns, structural frames and foundations, stairs and subparts, furniture, curtain wall mullions, cornices, and reveals. Each element shares a set of general identification properties, namely element ID, category, family, and type, following the taxonomy of Revit systems. Beyond these general fields, two additional groups are recorded: parameters and geometry. Parameters comprise category-specific presets and open dictionaries for family, type, and instance scopes. Geometry encodes the instantiation placeholder used during authoring (Listing 4).

In BIM authoring environments, building elements are instantiated with geometric and semantic configurations that vary across element classes. To handle this heterogeneity, the schema defines class-specific parameter presets that specify a compact set of parameters per element class while supporting forward-compatible extension. The current repository provides presets for 21 building element classes that are priorly mentioned (Listing 4), following the Revit category structures and their built-in parameters. Listing 5 presents an example of parameter JSON schema of wall elements. For walls, the preset captures the compound structure as a layer list with function with width, and material identification (compoundStructuralLayer), placement and orientation flags (flipped, orientation); whether the wall is stacked (isStackedWall); and curtain-wall grids if present (curtainGrid). It also records Revit built-in parameters as objects, including height (WALL_USER_HEIGHT_PARAM), width (WALL_ATTR_WIDTH_PARAM), area (HOST_AREA_COMPUTED), volume (HOST_VOLUME_COMPUTED), room-bounding (WALL_ATTR_ROOM_BOUNDING), construction and demolition phases (PHASE_CREATED, PHASE_DEMOLISHED), associated levels and offsets (WALL_BASE_CONSTRAINT, WALL_BASE_OFFSET, WALL_TOP_CONSTRAINT, WALL_TOP_OFFSET), association with center lines (WALL_KEY_REF_PARAM), and functions (WALL_ATTR_FUNCTION_PARAM, WALL_STRUCTURAL_SIGNIFICANT, WALL_STRUCTURAL_USAGE). The schema predefines properties that are critical for downstream estimation, as they influence quantities, boundary behavior, and constraints. Yet it also accommodates additional properties to ensure compatibility with evolving use cases.

To represent the geometry of building elements, this study formalizes the concept of geometric placeholders used during element instantiation, originally introduced in prior work [37]. These placeholders serve as minimal yet sufficient inputs to reconstruct authoring logic without storing full geometry, when combined with semantic information such as built-in parameters and editable property settings.

In Autodesk Revit, building elements are instantiated through object-oriented classes that reflect the physical hierarchy of the building. Each class encapsulates both geometric and parametric definitions, and the type of geometric placeholder varies depending on the element type and its instantiation context. For example, a wall may be instantiated using a single curve, or alternatively with a profile that includes openings. Similarly, a vertical column may be placed on a point, while sloped columns can be placed along a sloped curve. Listing 6 depicts the JSON schema used to formalize these geometric placeholders. The schema defines four major geometric types including *location point* (a point defined by X, Y, Z coordinates), *location curve* (a curve defined by points and geometric parameters, including specialized subclasses such as line,

Listing 1
Project-level BIM log JSON schema (project.schema.json).

project.schema.json

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://suhyungJang.github.io/revit-logger/schema/project.schema.json",
  "type": "object",  "additionalProperties": false,
  "required": ["projectGUID", "projectName", "sessions"],
  "properties": {
   "projectGUID": { "type": "string", "format": "uuid" }, "projectName": { "type": "string" },
   "renames": {
    "type": "array",
    "items": {
     "type": "object", "additionalProperties": false,
     "required": ["timestamp","oldName","newName"],
     "properties": {
      "timestamp": { "type": "string", "format": "date-time" }, "oldName": { "type": "string" }, "newName": { "type": "string" }
     }
    }
   },
   "sessions": {
    "type": "array",
    "items": { "$ref": "session.schema.json" }
   }
  }
}
```

Listing 2
Session-level BIM log JSON schema (session.schema.json).

session.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://suhyungJang.github.io/revit-logger/schema/session.schema.json",
 "type": "object", "additionalProperties": false,
 "required": ["sessionId","userName","startTime","events"],
 "properties": {
  "sessionId": { "type": "string" }, "userName": { "type": "string" }, "startTime": { "type": "string", "format": "date-time" }, "endTime": { "type": ["string","null"],
"format": "date-time" },
  "events": {
   "type": "array",
   "items": { "$ref": "event.schema.json" }
  }
 }
}
```

Table 3
Common data change types of BIM models (Lee et al., 2011) [47].

Type of BIM model data changes	Existence of objects		Property changes
	Previous model	Changed model	
CREATED	–	✓	N/A
DELETED	✓	–	N/A
PRESERVED	✓	✓	–
REVISED	✓	✓	✓

arc, cylindrical helix, ellipse, non-uniform rational B-splines, and Hermite spline), *curve loop* (a sequence of location curves forming a closed loop), and *profile* (a set of curve loops representing complex profiles). These types collectively represent building element geometries in Revit in a form suitable for capturing authoring intent and downstream use.

The state log is represented by a dedicated JSON schema that formalizes its role as the authoritative snapshot of the current model. The state log is an authoritative, up-to-date mapping of element IDs to their most recent semantic and geometric properties (Listing 7). Because the state is updated incrementally with each authoring event and only changed fields are written to event logs, the state log can be maintained with minimal overhead while exposing the full current model to downstream modules. Even for projects without prior logging, a state log can be initialized by extracting the desired information once from the current BIM model and then maintained incrementally thereafter while recording event logs.

3.2. Target value estimation module

The target value estimation module comprises two stages. First, a state log feature mapping stage converts the JSON-based state log into a fixed-dimensional, aggregated feature vector. Each feature is defined as an aggregation over a population of elements (e.g., counts, sums, averages) rather than as a raw list of element properties. This design allows the same state log to support various target value estimators by simply changing the feature configuration without modifying the logger itself. Second, a shape-constrained regression stage applies gradient-boosted tree models with monotonicity constraints and a t-dependent origin anchoring transformation. These constraints are designed to mitigate extrapolation violations that arise when models trained on finalized project datasets are deployed on early design authoring states.

3.2.1. State log feature mapping

The target value estimation module derives a fixed-layout input vector directly from the BIM state log. Rather than using raw element-level properties as model inputs, each feature is defined as an aggregation over a population of elements. This allows the same state log to support different performance targets simply by changing the feature configuration.

State log feature mapping follows the procedure in Listing 8. For each feature configuration ‘config,’ the field ‘config.path’ specifies where in the state log to look for candidate elements. The field ‘config.whereEquals’ is an optional set of equality conditions that all must hold for an element to be included. The field ‘config.value’ optionally names the property to read from each selected element. The field ‘config.aggregation’ specifies the aggregation operator, such as ‘count,’ ‘sum,’ ‘mean,’ ‘minimum,’ or ‘maximum’. Finally, ‘config.default’ stores the value to use when no element contributes.

In this algorithm, the selection path ‘config.path’ first collects the aggregation population from the state log. The equality filter ‘config.whereEquals’ then narrows this population to only those elements that satisfy all specified field–value pairs. If the aggregation operator is count and no value field is given, the feature is simply the number of elements that remain after filtering. Otherwise, the procedure reads the property

Listing 3
Event-level BIM log JSON schema (event.schema.json).

event.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://suhyungJang.github.io/revit-logger/schema/event.schema.json",
 "type": "object", "additionalProperties": false,
 "required": ["eventId","timestamp","eventType"],
 "allOf": [{"if": {"properties": {"eventType": {"const": "DELETED"}}},"then": {},"else": {"required": ["element"]}}],
 "properties": {
  "eventId": { "type": "integer"}, "timestamp": { "type": "string", "format": "date-time" }, "eventType": { "type": "string", "enum":
["CREATED","REVISED","DELETED"] },
  "element": { "$ref": "element.schema.json" }
 }
}
```

Listing 4
Element-level BIM log JSON schema (element.schema.json).

element.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://suhyungJang.github.io/revit-logger/schema/element.schema.json",
 "type": "object", "additionalProperties": false,
 "required": ["general"],
 "properties": {
  "general": {
   "type": "object", "additionalProperties": false,
   "required": ["elementId", "category", "family", "type"],
   "properties": {
    "elementId": {"type": "string"},
    "category": {"type": "string", "enum": ["OST_Levels", "OST_Rooms", "OST_Grids", "OST_Walls", "OST_Floors", "OST_Roofs", "OST_Ceilings", "OST_Windows", "OST_Doors", "OST_Columns", "OST_StructuralFraming", "OST_StructuralColumns", "OST_StructuralFoundation", "OST_Stairs", "OST_StairsRuns", "OST_StairsLandings", "OST_StairsRailing", "OST_Furniture", "OST_CurtainWallMullions", "OST_Cornices", "OST_Reveals"]},
    "family": {"type": "string"},
    "type": {"type": "string"}
   }
  },
  "parameters": {
   "type": "object", "additionalProperties": false,
   "properties": {
    "category": {"oneOf": [{"$ref": "categories/OST_Ceilings.schema.json"}, {"$ref": "categories/OST_Columns.schema.json"}, {"$ref": "categories/OST_Cornices.schema.json"}, {"$ref": "categories/OST_CurtainWallMullions.schema.json"}, {"$ref": "categories/OST_Doors.schema.json"}, {"$ref": "categories/OST_Floors.schema.json"}, {"$ref": "categories/OST_Furniture.schema.json"}, {"$ref": "categories/OST_Grids.schema.json"}, {"$ref": "categories/OST_Levels.schema.json"}, {"$ref": "categories/OST_Reveals.schema.json"}, {"$ref": "categories/OST_Roofs.schema.json"}, {"$ref": "categories/OST_Rooms.schema.json"}, {"$ref": "categories/OST_Stairs.schema.json"}, {"$ref": "categories/OST_StairsLandings.schema.json"}, {"$ref": "categories/OST_StairsRailing.schema.json"}, {"$ref": "categories/OST_StairsRuns.schema.json"}, {"$ref": "categories/OST_StructuralColumns.schema.json"}, {"$ref": "categories/OST_StructuralFoundation.schema.json"}, {"$ref": "categories/OST_StructuralFraming.schema.json"}, {"$ref": "categories/OST_Walls.schema.json"}, {"$ref": "categories/OST_Windows.schema.json"}]},
    "family": {"type": "object", "additionalProperties": {"oneOf": [{"type": "string"}, {"type": "number"}, {"type": "boolean"}, {"type": "null"}]}},
    "type": {"type": "object", "additionalProperties": {"oneOf": [{"type": "string"}, {"type": "number"}, {"type": "boolean"}, {"type": "null"}]}},
    "instance": {"type": "object", "additionalProperties": {"oneOf": [{"type": "string"}, {"type": "number"}, {"type": "boolean"}, {"type": "null"}]}}
   }
  },
  "geometry": {"$ref": "geometry.schema.json"}
 }
}
```

Listing 5
Wall element parameter JSON schema (categories/OST_Walls.schema.json).

categories/OST_Walls.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://Jang.github.io/revit-logger/schema/categories/OST_Walls.schema.json",
 "type": "object",
 "properties": {
  "compoundStructureLayer": {"type": "array", "items": {"type": "object", "properties": {"Function": {"type": "string"}, "Width": {"type": "number"}, "MaterialId": {"type": "integer"}, "MaterialName": {"type": "string"}}}},
  "flipped": {"type": "boolean"},
  "orientation": {"type": "string"},
  "isStackedWall": {"type": "boolean"},
  "curtainGrid": {"type": "string"},
  "builtIn": {"type": "object", "properties": {"WALL_USER_HEIGHT_PARAM": {"type": "number"}, "WALL_ATTR_WIDTH_PARAM": {"type": "number"}, "HOST_AREA_COMPUTED": {"type": "number"}, "HOST_VOLUME_COMPUTED": {"type": "number"}, "WALL_ATTR_ROOM_BOUNDING": {"type": "boolean"}, "PHASE_CREATED": {"type": "integer"}, "PHASE_DEMOLISHED": {"type": "integer"}, "WALL_BASE_CONSTRAINT": {"type": "integer"}, "WALL_BASE_OFFSET": {"type": "number"}, "WALL_TOP_CONSTRAINT": {"type": "integer"}, "WALL_TOP_OFFSET": {"type": "number"}, "WALL_KEY_REF_PARAM": {"type": "integer"}, "WALL_ATTR_FUNCTION_PARAM": {"type": "integer"}, "WALL_STRUCTURAL_SIGNIFICANT": {"type": "integer"}, "WALL_STRUCTURAL_USAGE": {"type": "integer"}}, "additionalProperties": true}
 }
}
```

named in ‘config.value’ from each remaining element, removes missing values, and applies the operator given in ‘config.aggregation’. If no valid value is available after filtering, the procedure falls back to ‘config.default’. The features produced in this way are appended to ‘featureVector’ in configuration order, yielding a fixed-layout input vector for the performance target.

Because each feature is defined as an aggregation, the module can express a wide range of inputs in a uniform way. For example, a feature *total wall area* uses a configuration whose path selects all elements, whose filter keeps only those with a wall category, whose value field points to an area parameter, and whose aggregation operator is sum. A second feature *number of walls* uses the same selection and filter but sets the aggregation operator to count and omits the value field, so the result is the count of wall elements. A third feature *average floor height* selects floor elements, reads a height-related quantity such as the difference between base and top elevations, and applies the mean operator. In all

Listing 6
Geometry JSON schema (geometry.schema.json).

geometry.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://suhyung-research.github.io/revit-log-json-schema/geometry.schema.json",
 "type": "object",
 "properties": {
  "geometryType": {"type": "string", "enum": ["locationPoint", "locationCurve", "curveLoop", "profile"]},
  "geometryParameters": {"oneOf": [{"$ref": "#/$defs/locationPoint"}, {"$ref": "#/$defs/locationCurve"}, {"$ref": "#/$defs/curveLoop"}, {"$ref": "#/$defs/profile"}]}
 },
 "$defs": {
  "locationPoint": {"type": "object", "properties": {"x": {"type": "number"}, "y": {"type": "number"}, "z": {"type": "number"}}, "required": ["x", "y", "z"]},
  "locationCurve": {
   "type": "object",
   "properties": {
    "line": {"type": "object", "properties": {"endPoint1": {"$ref": "#/$defs/locationPoint"}, "endPoint2": {"$ref": "#/$defs/locationPoint"}}, "required": ["endPoint1", "endPoint2"]},
    "arc": {"type": "object", "properties": {"plane": {"type": "string"}, "radius": {"type": "number"}, "startAngle": {"type": "number"}, "endAngle": {"type": "number"}}, "required": ["plane", "radius", "startAngle", "endAngle"]},
    "cylindricalHelix": {"type": "object", "properties": {"basePoint": {"$ref": "#/$defs/locationPoint"}, "radius": {"type": "number"}, "xVector": {"$ref": "#/$defs/locationPoint"}, "zVector": {"$ref": "#/$defs/locationPoint"}, "pitch": {"type": "number"}, "startAngle": {"type": "number"}, "endAngle": {"type": "number"}}, "required": ["basePoint", "radius", "xVector", "zVector", "pitch", "startAngle", "endAngle"]},
    "ellipse": {"type": "object", "properties": {"center": {"$ref": "#/$defs/locationPoint"}, "xRadius": {"type": "number"}, "yRadius": {"type": "number"}, "xAxis": {"$ref": "#/$defs/locationPoint"}, "yAxis": {"$ref": "#/$defs/locationPoint"}, "startParameter": {"type": "number"}, "endParameter": {"type": "number"}}, "required": ["center", "xRadius", "yRadius", "xAxis", "yAxis", "startParameter", "endParameter"]},
    "nurbsSpline": {"type": "object", "properties": {"degree": {"type": "integer"}, "knots": {"type": "array", "items": {"type": "number"}}, "controlPoints": {"type": "array", "items": {"$ref": "#/$defs/locationPoint"}}, "weights": {"type": "array", "items": {"type": "number"}}}, "required": ["degree", "knots", "controlPoints", "weights"]},
    "hermiteSpline": {"type": "object", "properties": {"controlPoints": {"type": "array", "items": {"$ref": "#/$defs/locationPoint"}}, "periodic": {"type": "boolean"}, "tangents": {"type": "array", "items": {"$ref": "#/$defs/locationPoint"}}}, "required": ["controlPoints", "periodic", "tangents"]}
   }
  },
  "curveLoop": {"type": "array", "items": {"$ref": "#/$defs/locationCurve"}},
  "profile": {"type": "array", "items": {"$ref": "#/$defs/curveLoop"}}
 }
}
```

Listing 7
State log JSON schema (stateLog.schema.json).

stateLog.schema.json

```
{
 "$schema": "https://json-schema.org/draft/2020-12/schema",
 "$id": "https://suhyung-research.github.io/revit-log-json-schema/stateLog.schema.json",
 "type": "object",
 "additionalProperties": false,
 "required": ["projectGUID", "timestamp", "projectName", "elements"],
 "properties": {
  "projectGUID": {"type": "string", "format": "uuid"},
  "timestamp": {"type": "string", "format": "date-time"},
  "projectName": {"type": "string"},
  "elements": {
   "type": "array",
   "items": {"$ref": "element.schema.json"}
  }
 }
}
```

three cases, the model receives a single scalar that summarizes a subset of elements, rather than a variable-length list of individual element records.

The state log is stored as JSON and can contain nested, multi-level structures (Supplementary material Data 1). To remain robust to such variation, the selection path is anchored at stable containers such as the top-level element list, and more detailed fields are only accessed when reading the value or applying the equality filter. Once a set of feature configurations is defined, the module can apply Listing 8 to any state-log snapshot and obtain aggregated inputs immediately after each authoring update, without re-parsing the BIM model.

In the following validation, input features used to train estimators targeting total project cost and construction duration were selected from quantitative features previously used in BIM-based quantity takeoff and early-stage estimation. Rather than adopting all features reported in prior studies, this study retained only those that can be reliably represented and incrementally updated during ongoing early design authoring. As discussed in Section 2.2, early BIM states generally lack sufficiently resolved material specifications, component assemblies, construction methods, and detailed classifications. Accordingly, features dependent on such construction-resolved information, including material- or assembly-specific quantities, were excluded. Instead, the selected

Listing 8
State log feature mapping from the BIM state log.

mapStateLogToFeatureVector(stateLog, featureConfigs)

```
function mapStateLogToFeatureVector(stateLog, featureConfigs):
    featureVector = []

    for config in featureConfigs:        # config: path, whereEquals, value, aggregation, default
        # 1. Select population from state log
        items = selectByPath(stateLog, config.path)

        # 2. Apply equality filter (if any)
        if config.whereEquals is not empty:
            items = [
                x for x in items
                if all(getField(x, k) == v for (k, v) in config.whereEquals)
            ]

        # 3. Aggregate
        if config.aggregation == "count" and config.value is None:
            value = len(items)
        else:
            values = [getField(x, config.value) for x in items]
            values = [v for v in values if v is not None]

            if values is empty:
                value = config.default
            else:
                value = applyAggregation(config.aggregation, values)  # e.g., summation, average, minimum, maximum

        featureVector.append(value)

    return featureVector
```

features consist of relatively stable primitive geometric and quantity-based properties (Table 4), including counts, surface areas, lengths, and heights for walls, slabs, columns, apertures (i.e., windows and doors), stairs, and building stories, including separate indicators for above-ground and underground stories. Although volumetric or assembly-specific features may improve estimation when finalized BIM models are available, the retained features provide a more consistent information basis for intermediate early-design states. These project-level aggregated properties are used as inputs to the target value estimation module to estimate the total construction cost and duration associated with each BIM authoring state.

3.2.2. *t-dependent origin anchoring transformation*

The proposed framework can employ various machine learning (ML) regressors for cost and duration estimation. However, standard regressors alone do not guarantee logically consistent behavior, when they are evaluated on unseen input feature distributions. In many practical settings, estimation during BIM authoring must be learned from datasets that contain only one final snapshot per project (i.e., the as-delivered BIM model) and its associated ground-truth project outcomes (e.g., total cost and duration). Intermediate authoring states are rarely recorded or labeled. Consequently, supervised models are trained almost exclusively on high-count, "completed" configurations of quantity-type features such as element counts and aggregated areas. When such models are deployed during BIM authoring, they are queried on early or partial BIM states whose feature distributions lie far below the ranges seen during training. Importantly, these early states correspond not merely to small values along individual input dimensions, but to a different region of the joint feature space that is weakly constrained by training data. Fig. 6 illustrates this behavior using a conventional regressor, a categorical boosting (CatBoost) [48] regressor trained on final-state element aggregates and total project costs. To approximate the deployment scenario, simulated BIM authoring sequences are constructed by extracting all relevant elements from each completed model, randomly ordering them, and incrementally adding them to form a sequence of intermediate BIM states (see Section 4.3.1 for detail). At each step, the aggregated input features extracted from the BIM state are utilized to conduct cost estimations as represented as the black line.

Table 4
Distributions of BIM-derived input features used for total construction cost and duration estimation.

Building element type		Properties	Unit	Minimum	Mean	Maximum
Wall		count	ea	52	542	2474
		$area_{sum}$	m^2	800	7529.0	25,889.0
Slab		count	ea	3	8	20
		$area_{sum}$	m^2	656.2	7609.9	33,745.0
Column		count	ea	0	51	730
		$length_{sum}$	mm	0.0	195.1	2550.3
Aperture		count	ea	30	317	1130
		$area_{sum}$	m^2	103.3	1387.9	6215.2
Stairs		count	ea	0	9	37
Building story	Above ground	count	ea	1	6	13
		$height_{avg}$	m	3	0.0	5.0
	Under ground	count	ea	3	1	3
		$height_{avg}$	m	3	4.1	10.8

Within where the number of elements is larger than 500, the estimated cost follows approximately monotonic increasing trends. In contrast, when the regressor is applied in the low-count states with the number of elements less than 500, two types of extrapolation violations are observed.

First, positive monotonicity is violated. In the simulated BIM authoring, quantity-type inputs (e.g., total wall count, total aperture area) increase monotonically as elements are only added in the simulation. Estimated cost should therefore be non-decreasing along the sequence. However, the estimated cost during low-count distribution in Fig. 6, depicts a consistent drop as the number of elements increases. This clearly indicates that a standard regressor can produce trend-inverting extrapolation under unseen input distributions.

Second, origin anchoring is violated. When no or very small number of elements present at a design state, the estimated c should be close to zero. However, standard regressors trained only on final-state projects

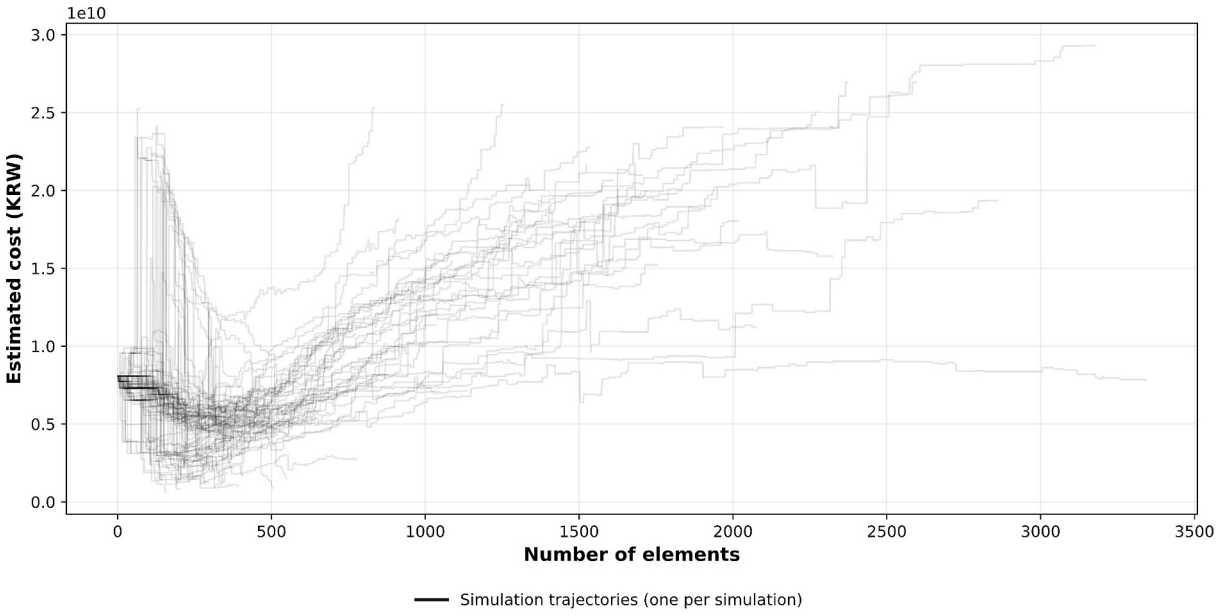


Fig. 6. Extrapolation behavior of a standard conventional cost estimator across varying input scales during simulated BIM authoring.

can exhibit a large positive baseline in early states. In Fig. 6, the estimated cost at the empty (or near-empty) state is already unrealistically high with over 0.75 billion Korean Won (KRW) (≈0.52 million USD), indicating that the model fails to anchor at the origin in unseen input distribution.

A possible mitigation strategy can be data augmentation to synthetically populate low-count regions. However, linear or rule-based augmentation requires element-level marginal cost and duration assumptions (or pre-defined unit-price/duration mapping), while these marginal impacts vary across projects. Imposing such synthetic structure can introduce another bias and degrade in performance.

To address these issues, this paper proposes a t-dependent origin anchoring transformation method. Here, each BIM authoring state is represented by an input feature vector x computed from aggregated quantity-type features. From this input, we define a scalar authoring-progress proxy $t(x) \in [0,1]$ that measures how far the current state lies between an empty configuration and the lower boundary of the seen training distribution. Let $Q \subseteq M$ denote the subset of monotone quantity features that characterize the scale of a partial BIM state (e.g., wall counts, slab areas, aperture areas, story counts). For each feature $j \in Q$, we compute a feature-wise reference value τ_j from a lower percentile of the training data (e.g., the 50th percentile). Because the proposed models are trained predominantly on final-state BIM snapshots, these reference values act as buffered lower bounds rather than direct estimates of early-stage quantities, ensuring stable normalization when evaluating partial BIM states whose feature magnitudes fall outside the joint range observed during training. Using these reference values we define a raw size-based progress score as in Eq. (3):

$$r(x) = \frac{1}{|Q|}\sum_{j\in Q} min\left\{1, max\left\{0, \frac{x_j}{\tau_j}\right\}\right\} \tag{3}$$

which reflects the average normalized scale of the monotone quantity features.

To distinguish early-stage states from those that fall within the range of quantities reliably observed during training, we further normalize this raw score using the empirical distribution of $r(x)$ in the training set. Specifically, let r_{seen} denote a reference percentile of the distribution of $r(x)$, chosen to approximate the smallest joint feature scale at which the model has received effective supervision. Together, the feature-wise reference scales τ_j and the state-level threshold r_{seen} function as distribution-aware buffers separating early design-stage inputs from the effective training regime.

The final authoring-progress index is then defined as in Eq. (4):

$$t(x) = clip\left(\frac{r(x)}{r_{seen}}, 0, 1\right) \tag{4}$$

so that $t(x) = 0$ represents an empty state, and $t(x) = 1$ indicates that the input has reached or exceeded the lower boundary of the in-distribution regime.

Let $\hat{y}(x) = f(x)$ denote the raw estimation of the regressor. The t-dependent, origin anchored prediction $\tilde{y}(x)$ is obtained using a multiplicative transformation as in Eq. (5):

$$\tilde{y}(x) = t(x) \bullet \hat{y}(x) \tag{5}$$

This formulation explicitly encodes the requirement that estimates should start from the origin and grow with authoring progress. For inputs whose joint feature scale is well represented in the training data ($t(x) \approx 1$), the transformation reduces to an identity mapping and leaves the regressor output effectively unchanged. In contrast, for early or partial BIM states whose feature combinations fall below the effective training regime, the transformation smoothly suppresses baseline bias. Because the transformation scales predictions uniformly as a function of $t(x)$, it preserves the relative ordering induced by the underlying regressor at each progress level. The transformation is applied only at inference time, where the training procedure and objective of the monotone regressor remain unchanged.

Fig. 7 depicts the updated behavior after t-dependent origin anchoring transformation application. The upper subfigure depicts gradual projection of estimated cost into origin anchor until the seen input distribution is entered as input at $t(x) = 1$ with the reference percentile of 50% is applied to both the τ_j and r_{seen}. The lower subfigure depicts the amount of shift when t-dependent origin anchoring transformation is applied to the original estimation results. Accordingly, the transformation progressively suppresses the baseline offset observed in early authoring states and smoothly recovers the original regressor output once the input enters the seen training regime, where $t(x) \approx 1$. As a result, the estimated cost trajectories is anchored near the origin for low-count BIM states while remaining essentially unchanged for inputs

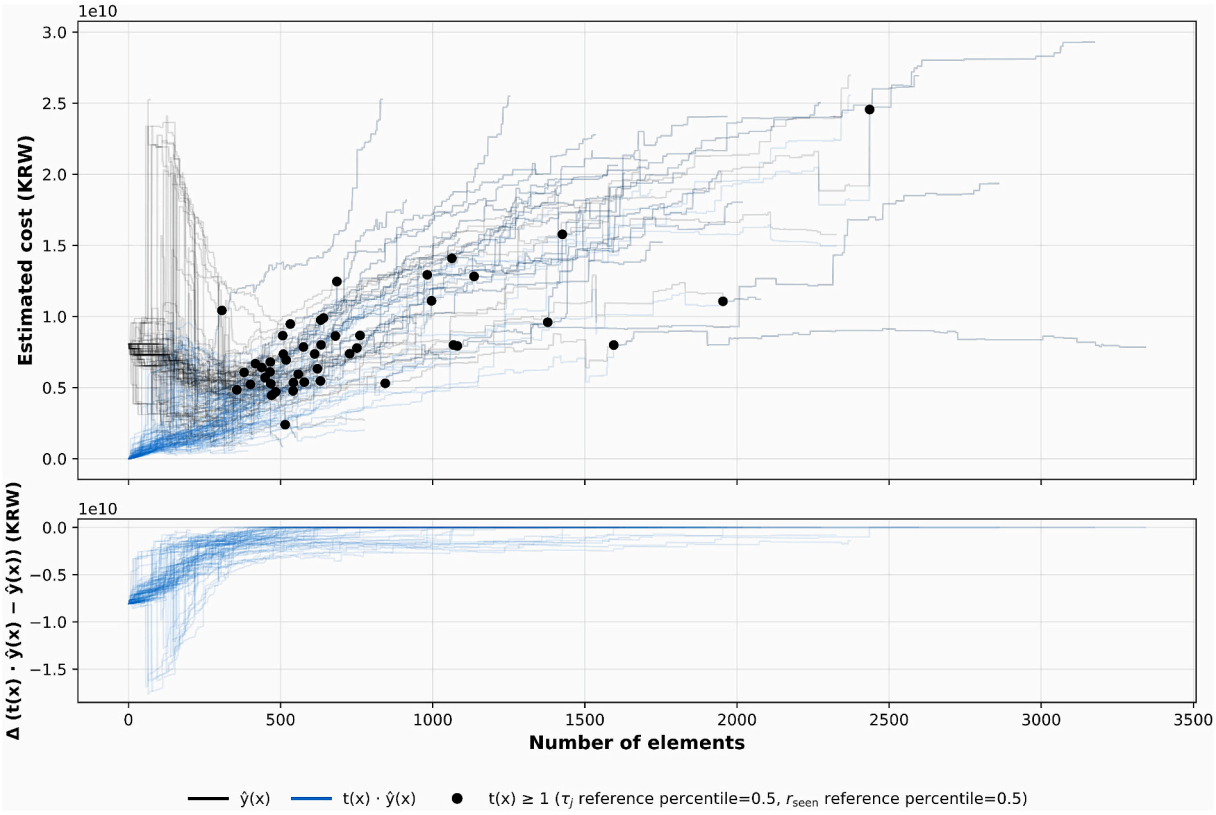


Fig. 7. Comparison between raw estimation of the regressor and estimation after t-dependent origin anchoring transformation.

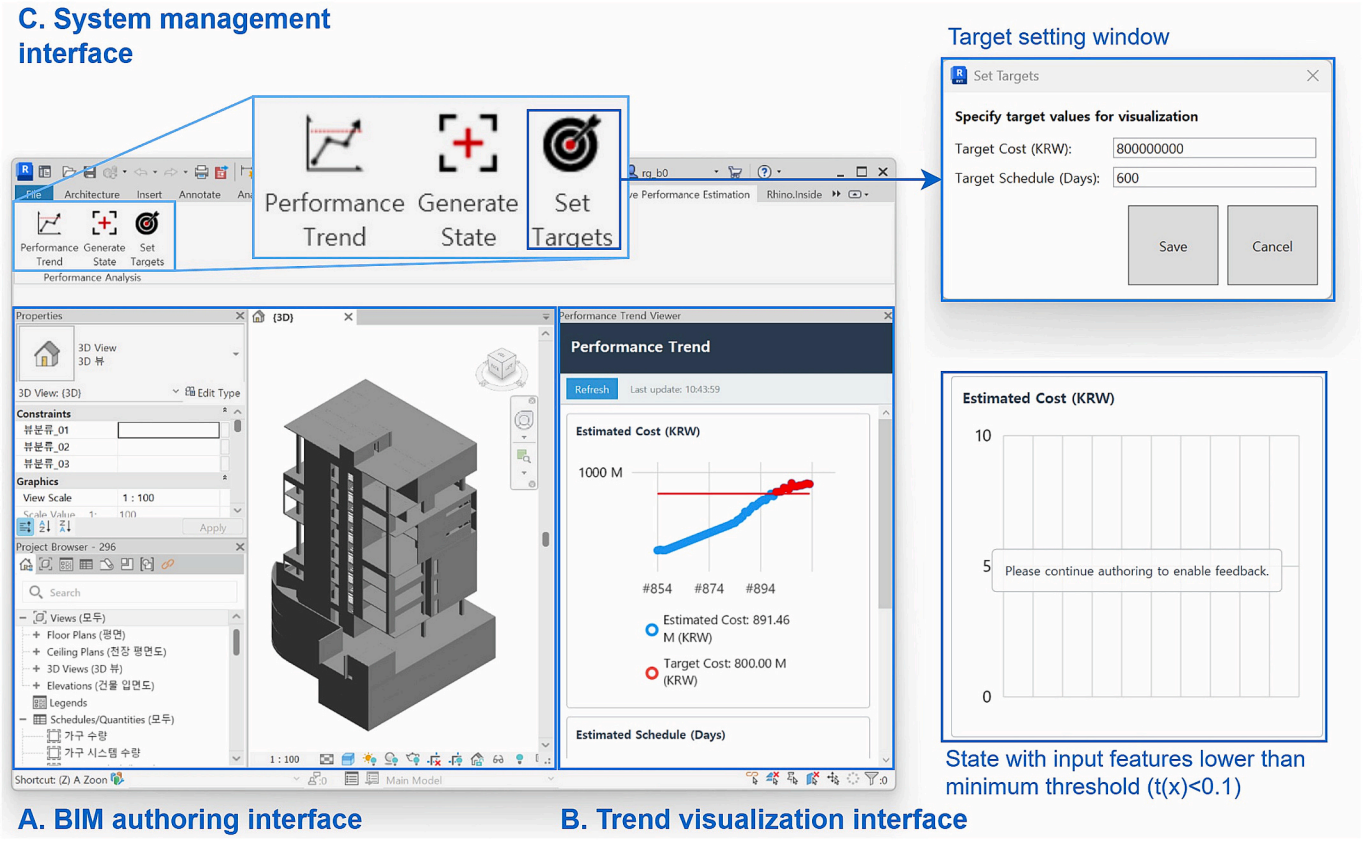


Fig. 8. Implemented interactive total cost and construction duration feedback framework as Revit add-in.

whose joint feature scales fall within the seen regime, thereby enforcing domain-consistent shape behavior without materially distorting the learned in-distribution mapping.

3.3. Trend visualization module

The trend visualization module provides continuous, event-aligned feedback on target value (e.g., total cost and construction duration) estimates during BIM authoring along with the targeted project performance. Each estimation produced by the target value estimation module is recorded together with its timestamp, project ID, and session ID, and organized as a chronological series. This enables tracking how the estimated cost and duration evolve over successive authoring actions.

The BIM authoring interface supports iterative creation and modification of building elements (Fig. 8A). As authoring actions occur, the current BIM state is passed to the target value estimation module, which produces updated cost and duration estimates reflecting the evolving design state.

The trend visualization interface is implemented as a dockable panel within the BIM environment (Fig. 8B). The panel plots estimated cost and estimated duration as time-series curves indexed by authoring events. Each data point corresponds to a discrete BIM state resulting from authoring actions (i.e., element creation, modification, or deletion). By aligning cost and duration trajectories with the actions, the panel enables users to observe how successive modeling decisions influence estimated cost and duration in real time. To avoid over-interpretation during extremely early authoring stages, the module suppresses visualization when the BIM state is too sparse to yield meaningful estimates. Specifically, estimation results are displayed only when the authoring state index satisfies a minimum threshold, $t(x) \geq t_{min}$ (we use $t_{min} = 0.1$ in this study). States with $t(x) < t_{min}$ are treated as a warm-up region and excluded from the plotted trajectory. During this phase, the panel displays a guidance message (e.g., “Please continue authoring to enable feedback”).

The system management interface is accessed through a dedicated graphical user interface (GUI) tab and provides three user actions (Fig. 8C): (1) opening the trend visualization panel, (2) generating an initial state log for models without prior logging history, and (3) defining target total cost and construction days. The initial state generation function supports workflows in which users start from an existing BIM model without previously recorded incremental logs. In this case, the system creates a baseline state log from the current model, after which subsequent cost and duration estimates are updated incrementally as new authoring events occur. The target-setting function allows users to specify cost and schedule objectives, such as a budget limit or expected completion deadline. These targets are visualized as reference lines overlaid on the estimated trajectories in the trend panel, enabling users to continuously assess whether the evolving design is trending toward, meeting, or exceeding predefined targets.

3.4. Implementation

The proposed interactive cost and duration feedback framework was implemented as an Autodesk Revit 2025 add-in on the Microsoft .NET platform. The three modules (event logging, target value estimation, and trend visualization) run within the same add-in, with the logging and estimation modules operating as back-end services, and the visualization module being exposed as a dockable user interface inside the BIM authoring environment.

For the event logging module, a context manager maintains project- and session-level log files and subscribes to Revit document and view events. Document-level events are used to manage the project–session hierarchy (creation, opening, saving, closing), and document-change events are used to record element-level creation, revision, and deletion. These API event arguments are mapped to the project, session, event, and element objects defined in the JSON schemas. Table 5 summarizes the mapping between the main Revit events and the internal logging managers. For each mutating event, the logger appends an event entry and applies the corresponding delta to the state log, keeping an up-to-date snapshot of all elements and their semantic and geometric properties.

Table 5
Mapping between Revit events and BIM log managers.

Manager	EventArgs (Autodesk.Revit.UI.Events)	Description
Element-level	.DocumentChangedEventArgs	Audits document changes and records element-level create, revise, and delete events.
Project/session context manager	.DocumentCreatedEventArgs	Audits creation of a new document and initializes project and session logs.
	.DocumentOpenedEventArgs	Audits opening of an existing document and starts a new session for the project.
	.DocumentSavingEventArgs	Audits document saving events and updates session metadata before writing logs.
	.DocumentClosingEventArgs	Audits document closing and finalizes the current session in the project log.
	.ViewActivatedEventArgs	Tracks active view changes to contextualize authoring actions within views.

For the target value estimation module, estimators were first trained offline in Python, then exported to the Open Neural Network Exchange (ONNX) format together with the corresponding feature definitions. For implementation, we utilized tree-based gradient boosting regressors, such as extreme gradient boosting (XGBoost) [49], light gradient boosting machine (LightGBM) [50], CatBoost, and histogram-based gradient boosting (HGBoost) [51], as base estimators. These tree-based gradient boosting regressors are known to be well suited for estimating cost and duration from BIM-derived tabular data that combines counts, geometric measures, and categorical attributes [52]. Detailed training settings are further explicated in Section 4.2. On the BIM environment deployment side, the Revit add-in is implemented in C# and uses the ‘Microsoft.ML.OnnxRuntime’ library to load the ONNX models once per session and keep them in memory. Whenever the state log is updated, the module aggregates the current BIM state into the fixed-layout input vector, calls the ONNX models through this runtime, and returns updated cost and duration estimates to the trend visualization module.

The trend visualization module was implemented as a dockable Windows Presentation Foundation (WPF) panel in Revit, using the LiveChartsCore library (SkiaSharp backend) to render interactive time-series plots of estimated target values.

4. Validation

Validation focuses on assessing whether the implemented interactive cost and duration feedback framework, especially the target value estimation module, delivers accurate and computationally efficient cost and duration estimation while addressing extrapolation violations that arises when using finalized project datasets to train as-authoring estimators. An ablation study was designed to analyze how the proposed t-dependent origin anchoring transformation, in combination with standard tree-based gradient boosting regressors, influences cost and duration estimation accuracy as well as extrapolation behavior on low-count authoring states (Fig. 9). All experiments, from machine learning model training to real-time evaluation, were conducted on a standard office notebook equipped with a Qualcomm Snapdragon X Elite CPU, an integrated Qualcomm Adreno GPU, a 16 GB RAM, and a 256 GB SSD, which satisfies the minimum system requirements for Autodesk Revit 2025.

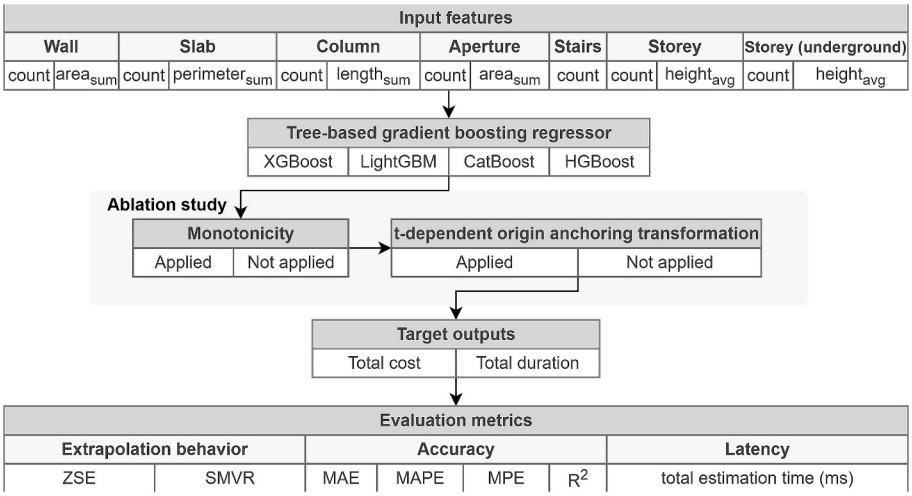


Fig. 9. Overview of ablation design, input features, target outputs, and evaluation metrics.

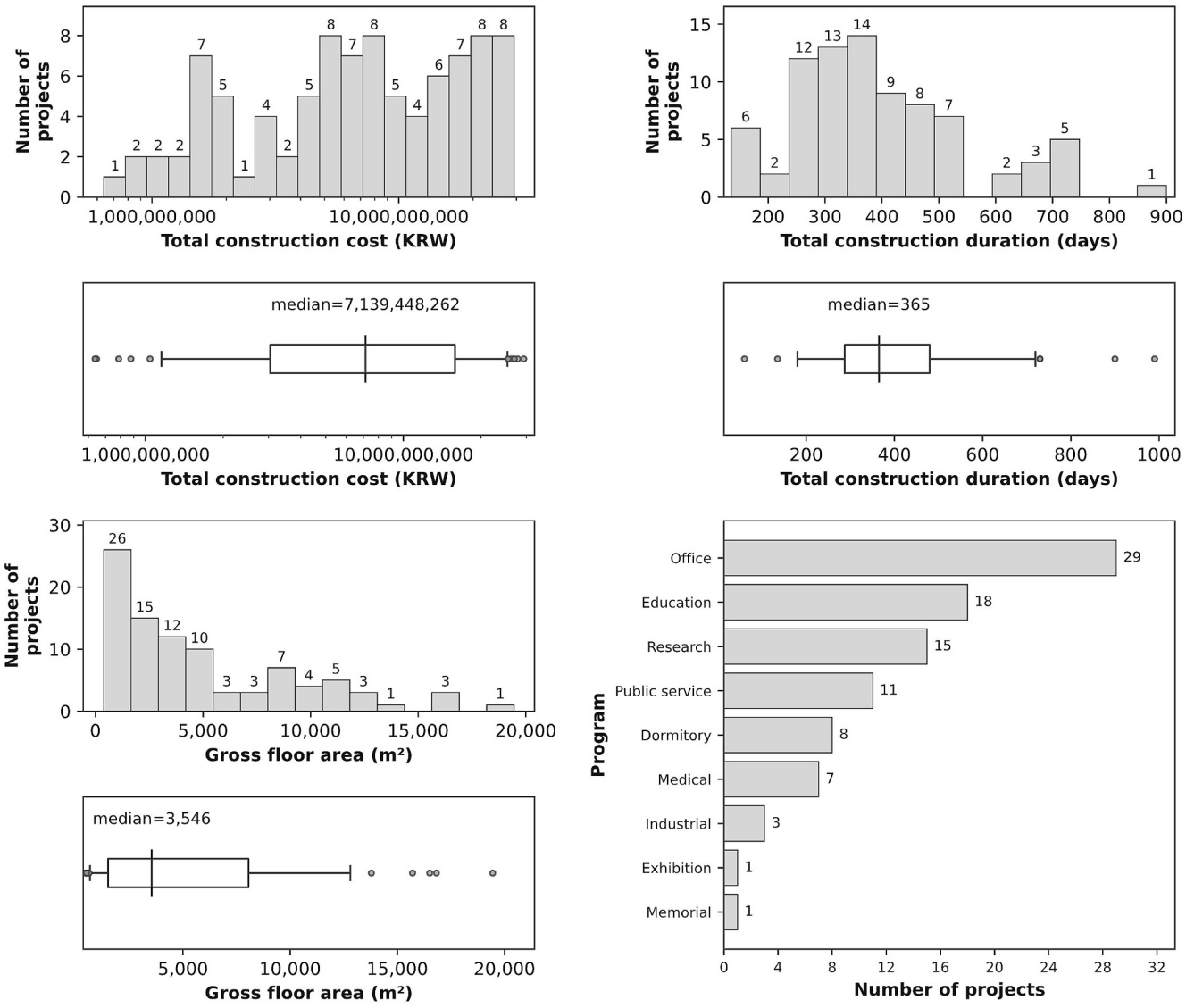


Fig. 10. Distributions of total construction cost, duration, gross floor area (GFA), and building programs in training dataset.

4.1. Dataset preparation

The training dataset consists of building element properties extracted from BIM models of built public projects registered in the Public Procurement Service (PPS) of the Republic of Korea, all of which use reinforced concrete as their primary structural system. A total of 93 BIM models were reconstructed from original construction drawings for public projects, spanning major PPS building programs (primarily office buildings and school facilities) across a range of project scales. All building elements were modeled to LOD 200, representing specified but not fully detailed dimensions with generic composition (i.e., without specified materials) following the centerlines of plan drawings. This level of abstraction aligns with the intended scope of the proposed framework.

For ground-truth labels used in training and testing, total construction cost and total construction duration disclosed by the Public Procurement Service of the Republic of Korea were paired with corresponding project BIM models. The cost target represents the reported total project-level cost, including direct and indirect construction-related costs, administrative expenses, and profit. The duration target represents the construction period from the reported commencement date to the completion date. Across the 93 projects, total construction cost ranges from approximately 0.64 to 29.3 billion KRW with median at 8.14 billion KRW. Total construction duration is available for 84 projects after excluding cases with missing or non-positive duration values. In this subset, the duration ranges from 60 to 990 days with median at 365 days. The GFA of the 93 projects ranges from 387.25 m^2 to 19,449.94 m^2. The building functions include offices (29 cases, 30.85%), educational buildings such as kindergartens and schools (18 cases, 19.15%), research facilities (15 cases, 15.96%), public service buildings such as police stations, fire stations, and post offices (11 cases, 11.70%), and others. The distributions of these labels are shown in Fig. 10, highlighting the substantial variability in project scale that the proposed framework must accommodate.

To form model inputs, element-level features are extracted from each BIM model following the definitions in Table 4. These features are aggregated to project-level representations and paired with the corresponding total construction cost or duration as continuous regression targets. Separate estimators are trained for cost and duration. Projects with missing or non-positive cost or duration values are excluded, leaving 93 projects (i.e., all projects) for cost estimation, and 84 for duration estimation. For each target, five-fold stratified cross-validation is used with an 80/20 split. Five strata are defined based on log-transformed total cost for cost estimation and raw duration in days for duration estimation to preserve balanced target distributions across folds.

4.2. Experimental design

The experiment evaluates estimation performance across different estimator backbones with ablation of shape-control strategies: (1) different tree-based gradient boosting regressors as estimator backbones, (2) the inclusion or exclusion of a direct monotonicity constraint to models, and (3) the inclusion or exclusion of the proposed t-dependent origin anchoring transformation. Four tree-based gradient boosting regressors (XGBoost, LightGBM, CatBoost, and HGBoost) are used as base models, allowing us to examine how the proposed transformation behaves across different inductive biases. XGBoost uses level-wise tree growth with strong regularization and therefore tends to learn globally stable average patterns in cost and duration across the project population. LightGBM uses leaf-wise growth with histogram-based optimization and aggressive sampling, which biases it toward capturing sharp local changes and tail behavior, such as sudden cost increases in very large or complex projects. CatBoost uses ordered boosting and target statistics for categorical features, which can reduce prediction shift and overfitting on small or imbalanced samples while preserving fine-grained interactions between project attributes and quantity features. HGBoost employs histogram-based splits with more conservative depth and leaf constraints, leading to smoother and more regular piecewise functions that are expected to be more stable under noisy or sparse early-stage design states. All models were implemented using standard Python libraries (‘xgboost.xgbregressor,’ ‘lightbgm.LGBMRegressor,’ ‘catboost.CatBoostRegressor,’ and ‘sklearn.ensemble.HistGradientBoostingRegressor’).

Hyperparameters were optimized in a fully automated and reproducible manner using Optuna Python library. In contrast to the original implementation, which relied on manual and iterative trial-and-error, this study adopts a systematic search strategy based on a Tree-structured Parzen Estimator (TPE) sampler combined with a median pruner. All optimizations were guided by the validation MAPE as the objective metric. Model-specific hyperparameters were tuned within ranges commonly recommended in prior work and official implementations. Fall all models, learning rate was optimized. For XGBoost, the search space included the number of estimators, tree depth, subsampling ratios, and child-weight regularization. LightGBM optimization focused on the number of boosting iterations, leaf complexity, subsampling, and minimum child sample constraints. For CatBoost, learning rate, number of iterations, tree depth, and L2 regularization strength were tuned. The HistGradientBoostingRegressor was optimized with respect to learning rate, number of boosting iterations, tree depth, leaf-node constraints, and minimum samples per leaf. The hyperparameter search spaces for each regressor model and the resulting optimized values are summarized in Table S1 at Supplementary material section.On top of these base models, two shape-related settings are varied for ablation. First, the proposed t-dependent origin anchoring transformation is either applied or omitted, allowing a direct comparison between raw predictions and transformed predictions for each regressor. This comparison indicates to what extent the transformation can reduce extrapolation errors on out-of-distribution input features without affecting in-distribution accuracy. The reference percentiles used to τ_j and the r_{seen} were both set to 50% in the experiment. Second, for tree-based gradient boosting models that support feature-wise monotonicity constraints, monotonicity can be enabled. In this case, the monotone features are defined from domain knowledge rather than inferred from data, and the constraint can be used either alone or in combination with the t-dependent transformation. For each combination of base model and constraint setting, separate models for cost and duration are trained under the same training, validation, and test splits, and both accuracy metrics and extrapolation behavior metrics on simulated authoring sequences are reported.

4.3. Evaluation metrics

The evaluation focused on three dimensions including extrapolation violations on low-count states (i.e., out-of-distribution from training dataset), estimation accuracy, and latency. This section introduces the metrics used for each of these dimensions and clarifies how they relate to the research objectives.

4.3.1. Low-count extrapolation violation metrics

As explained in Section 3.2.2, the cost and duration estimations should conform to positive monotonicity with respect to quantity-type features (e.g., counts, areas, or volumes), and should be anchored at zero when no relevant elements are represented in the design. To evaluate extrapolation behavior in low-count distributions, we generated a mock-up scenario of BIM authoring based on the final-state BIM models used in the training dataset. For each final-state BIM model, we first identified the building elements of interest (i.e., walls, slabs, columns, apertures, stairs, and building stories). We then simulated BIM authoring by randomly ordering these elements and adding them one by one. As each element was added, the aggregated input features were updated, and a new row was appended to the mock-up dataset. This procedure

yields a sequence of intermediate BIM states per project, enabling us to assess how models trained on final-state datasets behave under unseen input feature distributions corresponding to early and intermediate authoring states.

Two complementary metrics were designed to evaluate how well models satisfy these deployment-oriented shape requirements under as-authoring conditions, namely stepwise monotonicity violation rate (SMVR) and zero-state error (ZSE). ZSE quantifies how well the model respects origin anchoring at the earliest design state. Formally, ZSE measures the average magnitude of the prediction at the zero-state as Eq. (6):

$$\mathrm{ZSE} = \mathbb{E}(|\hat{y}(t)\,||t = 0\,) \tag{6}$$

where $\hat{y}(t)$ denotes the predicted cost or duration evaluated along the normalized authoring-progress index t, and $t = 0$ corresponds to the earliest mock-up state, in which no element inputs are present and all aggregate input features are zero (or the minimum observed value if an exact $t = 0$ state does not exist). Lower ZSE indicates better origin anchoring, because a well-anchored model should predict near-zero cost and duration when no elements have been modeled.

While ZSE focuses on the origin, models can still violate monotonicity as authoring progresses. To quantify such behavior, we define SMVR as a stepwise metric, which measures how often estimations decrease between consecutive authoring states despite an increase in modeled quantity.

For each project, we consider a sequence of BIM mock-up states $\{x_s\}_{s=1}^{S}$ ordered by increasing authoring progress, with quantity-type features monotonically increasing between consecutive states. Let $\hat{y}(x_s)$ denote the estimated cost or duration at state x_s. The SMVR is defined as the fraction of state transitions in which the predicted value decreases along the sequence as Eq. (7):

$$\mathrm{SMVR} = \frac{|\{s|\hat{y}(x_s) < \hat{y}(x_{s-1})\,, s = 2, \ldots, S\}|}{S-1} \times 100 \tag{7}$$

In this study, the mock-up authoring sequences are constructed under the simplifying assumption that information is never removed between consecutive states (i.e., elements are only added and not deleted), such that an increase in t corresponds to an increase in modeled quantity. Under this assumption, any decrease in the predicted cost or duration between consecutive states constitutes a monotonicity violation. Intuitively, SMVR therefore measures the fraction of incremental authoring steps for which the model produces counterintuitive decreases in estimation results, a behavior that is particularly problematic in low-count, early-stage extrapolation regimes.

SMVR is defined and computed stepwise over the entire mock-up trajectory, providing a global measure of monotonicity stability. However, because a single aggregated SMVR averages violations across all authoring stages, it does not indicate where along the authoring process such violations occur. In particular, violations confined to narrow progress ranges may be obscured when summarized by a single global rate. To address this limitation, we further analyze SMVR within the extrapolation regime defined by the authoring-progress index $t(x) < 1$, which corresponds to early and intermediate authoring states below the lower boundary of the seen training distribution. Within this regime, the same stepwise SMVR is reported separately across four quartiles of $t(x)$, based on its empirical distribution over all mock-up states. SMVR is thus evaluated within each quartile using only consecutive state pairs that fall into the same progress range. This quartile-wise analysis does not introduce a new metric, but conditions the stepwise SMVR on different regions of the extrapolation domain, enabling a stage-wise interpretation of monotonicity behavior and clarifying whether violations are localized to specific phases of low-count extrapolation or persist into later, better-supported stages of authoring.

Together, SMVR and ZSE provide a deployment-oriented view of shape behavior under extrapolation to unseen input distributions at deployment. ZSE evaluates how well the model respects origin anchoring at zero-input states, while SMVR captures how frequently monotonicity is violated as the BIM model grows. These metrics allow us to compare monotone and non-monotone models, as well as different zero-anchor configurations, under the same training data and base architectures. Furthermore, statistical significance was assessed using two-sided Wilcoxon signed-rank tests on paired project-level SMVR and ZSE values. Each configuration was compared against the baseline setting in which neither the monotonicity constraint nor the t-dependent origin anchoring transformation was applied.

4.3.2. Accuracy metrics

To evaluate estimation accuracy, we use standard regression metrics computed separately for cost and duration. The first metric is MAE, which summarizes the average absolute deviation between predictions and ground truth as Eq. (8):

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i| \tag{8}$$

where $n$is the number of samples, y_i is the true value for sample i, and $\hat{y}_i$ is the corresponding prediction. MAE is scale-dependent but less sensitive to large outliers than squared-error-based metrics. MAE is scale-dependent but less sensitive to large outliers than squared-error-based metrics. In our setting, MAE directly expresses the difference between estimation result and ground truth in the same units as the target (currency for cost and days for duration), depicting absolute cost and time deviations.

The second metric is the mean absolute percentage error (MAPE) as Eq. (9):

$$\mathrm{MAPE} = \frac{1}{n}\sum_{i=1}^{n}|\frac{y_i - \hat{y}_i}{y_i + \varepsilon}| \times 100 \tag{9}$$

where ε is a small positive constant added to the denominator to avoid division by zero when the ground-truth value y_i is close to zero. MAPE complements MAE by normalizing errors by project scale, which is important because the dataset contains projects with widely varying cost and duration magnitudes.

Third, we compute the mean percentage error (MPE) as Eq. (10):

$$\mathrm{MPE} = \frac{1}{n}\sum_{i=1}^{n}\frac{y_i - \hat{y}_i}{y_i + \varepsilon} \times 100 \tag{10}$$

which preserves the sign of the relative error. MPE is used to detect whether a model systematically overestimates or underestimates, which is critical for assessing the risk of persistent budget or duration bias.

For MAE, MAPE, and MPE, statistical significance was assessed using two-sided Wilcoxon signed-rank tests on paired project-level errors. Each configuration was compared against the baseline case in which neither the monotonicity constraint nor the t-dependent origin anchoring transformation was applied.

Finally, we report the coefficient of determination as Eq. (11):

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2} \tag{11}$$

where $\bar{y} = \frac{1}{n}\sum_{i=1}^{n} y_i$ is the meaning of the true values. R^2 provides a scale-independent summary of how much variation across projects the model explains beyond a mean baseline.

Table 6
Tested extrapolation constraint violations using stepwise monotonicity violation rate (SMVR) and zero-state error (ZSE) as metrics on mock authoring simulations.

Target project performance	Base regressor	t-dependent origin anchoring	Monotonicity constraint	SMVR (%) mean (standard deviation)	ZSE (KRW/days) mean (standard deviation)
Total cost	**XGBoost**	×	×	6.635 (±0.340)	10,333,432,832 (±6,190,171,584)
			✓	0.335 (±0.022) ***	365,576,742 (±162,311,126) ***
		✓	×	5.385 (±0.294) ***	0 (±0) ***
			✓	0.331 (±0.022) ***	0 (±0) ***
	LightGBM	×	×	0.965 (±0.169)	1,703,143,294 (±266,906,422)
			✓	0.216 (±0.035) ***	1,437,108,173 (±92,234,426) ***
		✓	×	0.883 (±0.163) ***	0 (±0) ***
			✓	0.207 (±0.034) ***	0 (±0) ***
	CatBoost	×	×	3.819 (±0.930)	5,475,850,665 (±2,939,606,693)
			✓	0.464 (±0.021) ***	446,500,206 (±57,048,867) ***
		✓	×	3.142 (±0.821) ***	0 (±0) ***
			✓	0.458 (±0.024) ***	0 (±0) ***
	HGBoost	×	×	0.239 (±0.161)	1,945,432,627 (±118,627,012)
			✓	0.124 (±0.054) ***	1,893,135,740 (±77,866,230) ***
		✓	×	0.176 (±0.080) ***	0 (±0) ***
			✓	0.115 (±0.046) ***	0 (±0) ***
Construction duration	**XGBoost**	×	×	5.571 (±0.235)	105.8 (±96.7)
			✓	0.364 (±0.010) ***	63.3 (±38.2) ***
		✓	×	4.619 (±0.174) ***	0.0 (±0.0) ***
			✓	0.358 (±0.009) ***	0.0 (±0.0) ***
	LightGBM	×	×	0.922 (±0.087)	185.4 (±23.8)
			✓	0.230 (±0.013) ***	181.5 (±12.5) ***
		✓	×	0.846 (±0.076) ***	0.0 (±0.0) ***
			✓	0.225 (±0.007) ***	0.0 (±0.0) ***
	CatBoost	×	×	5.763 (±0.495)	119.5 (±56.8)
			✓	0.389 (±0.008) ***	40.8 (±22.5) ***
		✓	×	4.615 (±0.183) ***	0.0 (±0.0) ***
			✓	0.369 (±0.011) ***	0.0 (±0.0) ***
	HGBoost	×	×	0.213 (±0.041)	235.4 (±9.8)
			✓	0.186 (±0.018) ***	235.4 (±9.7) ***
		✓	×	0.207 (±0.032) ***	0.0 (±0.0) ***
			✓	0.185 (±0.019) ***	0.0 (±0.0) ***

*✓ = monotonicity constraint / t-dependent origin anchoring transformation applied; × = not applied; *** = $p < 0.001$ based on the Wilcoxon signed-rank test, using the case in which neither the monotonicity constraint nor the t-dependent origin anchoring transformation was applied as the baseline.

4.3.3. Latency metrics

Latency metrics are used to assess whether the proposed framework can provide estimation with responsiveness compatible with interactive BIM authoring. Following prior human–computer interaction research [21], responses faster than 100 milliseconds are treated as perceptually immediate for direct manipulation, responses around 1000 milliseconds mark the boundary at which users begin to perceive an interruption, and responses around 10,000 milliseconds are often associated with the upper bound of a single unit task. These three values (100, 1000, and 10,000 milliseconds) are adopted as reference thresholds for real-time feedback.

Latency is measured directly from the mock-up authoring simulation described in Section 4.1. For each simulated event (element creation or update), we record total estimation time under two feature-extraction mechanisms: (1) the framework's state-log-based incremental updates, which recompute only features affected by the updated element, and (2) conventional BIM model-level extraction, which recomputes all features by scanning the entire model at each state. In both cases, total estimation time is defined as feature-extraction (or update) time plus a fixed average model inference time, since inference time does not materially depend on the authoring state. The comparison is made between conventional full-model extraction and logging-based incremental updates. Inference time is fixed using the slowest evaluated model to provide a conservative latency estimate.

Total estimation time is recorded for every simulated authoring event across all 93 BIM models. For each project and mechanism, we identify the first event at which total latency exceeds each threshold (100, 1000, and 10,000 milliseconds) and record the corresponding number of modeled elements. Results are then aggregated across projects to characterize typical model sizes at which each threshold is violated and to compare how latency grows under incremental logging versus full-model extraction.

5. Results

This section presents the validation results of the proposed framework in three dimensions. First, the extrapolation behavior of the estimators under early, low-count BIM authoring states is evaluated using the SMVR and ZSE. Second, the effects of the proposed t-dependent origin anchoring transformation and monotonicity constraints on cost and duration estimation accuracy are examined using MAE, MAPE, MPE, and R^2. Finally, the computational latency of the framework is assessed to determine its effectiveness for interactive BIM authoring.

5.1. Low-count extrapolation violation

To evaluate extrapolation behavior on early, low-count authoring states, the SMVR and the ZSE are analyzed along the simulated authoring sequences. SMVR measures how often predictions decrease when quantity-type features increase between successive simulated authoring states, while ZSE measures the deviation of cost or duration estimates at empty or nearly empty BIM states from the desired origin (i.e., a prediction close to zero). Four configurations are compared for each base regressor: (1) no monotonicity constraint and no t-dependent origin anchoring transformation, (2) monotonicity constraint only, (3) t-dependent origin anchoring transformation only, and (4) both

Table 7
Tested accuracy for total cost estimation under monotonicity constraints.

Base regressor	t-dependent origin anchoring	Monotonicity constraint	MAE (KRW) mean (standard deviation)	MAPE (%) mean (standard deviation)	MPE (%) mean (standard deviation)	R^2 mean (standard deviation)
XGBoost	×	×	2,230,134,013 (±792,597,734)	46.1 (±32.1)	−18.6 (±38.1)	0.772 (±0.214)
		✓	2,279,998,032 (±914,990,734)	41.9 (±21.6)	−16.5 (±25.1)	0.745 (±0.239)
	✓	×	**2,237,117,551 (±809,253,732)**	**44.4 (±28.4)**	**−14.8 (±35.2)** ***	**0.761 (±0.237)**
		✓	2,295,111,126 (±920,201,206)	42.3 (±20.7)	−14.3 (±24.0)	0.740 (±0.247)
LightGBM	×	×	3,335,143,220 (±600,545,942)	58.4 (±28.2)	−25.7 (±36.0)	0.615 (±0.198)
		✓	2,749,558,232 (±877,340,640) **	49.2 (±31.2) **	−20.9 (±34.5)	0.695 (±0.210)
	✓	×	**3,339,699,065 (±620,504,451)**	**57.1 (±25.5)**	**−22.2 (±34.7)** ***	**0.608 (±0.210)**
		✓	2,755,098,655 (±885,997,124) **	47.9 (±27.9) **	−17.4 (±32.6)	0.687 (±0.222)
CatBoost	×	×	2,457,558,481 (±612,250,382)	44.0 (±21.8)	−18.2 (±27.6)	0.736 (±0.185)
		✓	3,121,338,171 (±752,815,936) ***	39.5 (±12.7)	7.5 (±19.5) ***	0.620 (±0.189)
	✓	×	**2,438,392,416 (±638,269,211)**	**41.2 (±18.6)**	**−14.1 (±25.2)** ***	**0.731 (±0.197)**
		✓	3,141,926,653 (±763,857,981) ***	40.4 (±12.0) *	9.8 (±18.8) ***	0.616 (±0.192)
HGBoost	×	×	2,697,862,887 (±515,804,963)	48.5 (±30.6)	−19.4 (±33.8)	0.708 (±0.155)
		✓	2,663,110,101 (±564,800,944)	47.6 (±30.5) *	−18.5 (±33.8) *	0.713 (±0.161)
	✓	×	**2,687,763,884 (±521,749,144)**	**46.0 (±27.7)**	**−15.2 (±31.9)** ***	**0.702 (±0.170)**
		✓	2,653,095,039 (±571,056,809)	45.1 (±27.6) **	−14.3 (±31.8) ***	0.706 (±0.175)

* *✓ = monotonicity constraint / t-dependent origin anchoring transformation applied; × = not applied; * = $p < 0.05$, ** = $p < 0.01$, and *** = $p < 0.001$ based on the Wilcoxon signed-rank test, using the case in which neither the monotonicity constraint nor the t-dependent origin anchoring transformation was applied as the baseline.

monotonicity constraint and t-dependent origin anchoring transformation (Table 6). The results show that regressors trained solely in final-state BIM models exhibit extrapolation violations when applied to early, low-count authoring states.

Even without any shape control (i.e., monotonicity constraint and t-dependent origin anchoring), LightGBM and HGBoost exhibit largely positive monotonic behavior, with mean SMVR remaining in the range of approximately 0.2–1.0% for both total cost and construction duration estimation. Applying both shape control strategies in these cases further reduces SMVR to the range of 0.1 to 0.2%. XGBoost and CatBoost show substantially higher violation rates, with mean SMVR reaching approximately 3.8–6.6% for both cost and duration estimation, when no shape control is applied. With shape control strategies, these models also achieve mean SMVR levels comparable to those of LightGBM and HGBoost, decreasing to approximately 0.33–0.45%. Overall, both shape control strategies, whether applied individually or in combination, consistently reduce mean SMVR across all base regressors with statistical significance ($p < 0.001$ based on the Wilcoxon signed-rank test). Among them, the explicit monotonicity constraint depicts the higher effectiveness in lowering the mean SMVR values.

In contrast to SMVR, the ZSE highlights the role of the t-dependent origin anchoring transformation in satisfying origin anchoring requirements under extrapolation. In the unconstrained setting (without monotonicity constraints and without t-dependent origin anchoring), mean ZSE is extremely large for both cost and duration, particularly for cost estimation. For example, the unconstrained XGBoost cost model yields a mean ZSE of approximately 10.3 billion KRW with standard deviation of 6.1 billilon KRW. This mean value is comparable to the one-third of the highest final project cost observed in the training distribution (approximately 29.3 billion KRW) and the standard deviation is almost equivalent to the median cost in the training distribution (approximately 7.1 billion KRW), despite being evaluated at empty or nearly empty BIM states. While the monotonicity constraint and t-dependent origin anchor transformation depicts statistical significance ($p < 0.001$) in reducing mean ZSE, mean ZSE for costs and duration remains larger than 0.36 billion KRW and 40 days when only monotonicity constraints is applied. By contrast, in all configurations where the t-dependent origin anchoring transformation is applied, mean ZSE is fixed at zero for both cost and duration across all base regressors. This result confirms that the t-dependent origin anchoring transformation is uniquely effective in enforcing correct origin alignment under low-count extrapolation.

Overall, the results show that both monotonicity constraints and the t-dependent origin anchoring transformation significantly suppress SMVR and ZSE. However, using monotonicity constraints alone do not guarantee low ZSE, depicting unrealistically large cost or duration values can be estimated from near-empty BIM states during early design stage. In contrast, the t-dependent origin anchoring transformation eliminates ZSE entirely by fixing the zero-state estimation at the origin.

5.2. Accuracy

The results of accuracy evaluation show that the proposed t-dependent origin anchoring transformation not only reduces extrapolation errors, as demonstrated in the previous analysis, but also improves estimation accuracy (Table 7). These improvements are observed across

all tested base regressors, in contrast to feature-wise monotonicity constraints, whose effect on accuracy is model-dependent and often detrimental.

Applying monotonicity constraints significantly degrades accuracy for CatBoost and XGBoost. For CatBoost, the negative impact is more pronounced with mean MAE decreased at $p < 0.001$ and mean R^2 decreases by more than 0.1. In contrast, the histogram-based models (i.e., LightGBM and HGBoost) show improved accuracy under monotonicity constraints. In LightGBM, mean R^2 increases from 0.615 to 0.695. HGBoost also shows a slight improvement, with mean R^2 increasing from 0.708 to 0.713. These results suggest that monotonicity constraints can act as either a helpful and harmful inductive bias depending the regressor architecture. In contrast, the t-dependent origin anchoring transformation shows consistent accuracy improvements when applied without monotonicity constraints across all four regressor with mean MPE improvements at $p < 0.001$, while no significant decrease observed in the other evaluation metrics.

For construction duration estimation, a similar overall trend is observed (Table 8). The proposed t-dependent origin anchoring transformation consistently improves MPE relative to the raw models, whereas the effect of monotonicity constraints remains inconsistent. As in the cost estimation results, CatBoost shows severe performance degradation under monotonicity constraints, with mean R^2 decrease below zero both without (−1.537) and with (−1.559) t-dependent anchoring transformation. MAE also increases from about 70 days to more than 210 days. For duration estimation, monotonicity constraints do not produce any significant positive effect for LightGBM or HGBoost. Again, when applied without monotonicity constraints, the t-dependent origin anchoring transformation consistently improves duration estimation accuracy across all four regressors, with significant improvements in mean MPE at $p < 0.01$.

Overall, the accuracy results show that the effect of monotonicity constraints depends strongly on the regressor architecture and can be either positive or negative. In some cases, especially for CatBoost, the constraint leads to substantial performance degradation. By contrast, the t-dependent origin anchoring transformation consistently improves bias-related accuracy across both cost and duration estimation, particularly in MPE, without significantly decreasing the other evaluation metrics. This suggests that the proposed t-dependent origin anchoring transformation offers a stable and broadly applicable way to improve estimation accuracy while maintaining meaningful extrapolation behavior near the lower boundary of the training distribution.

5.3. Latency

Latency was evaluated by combining model inference time with feature extraction time along simulated BIM authoring sequences. Feature extraction was measured under two scenarios: (1) the proposed state-log-based incremental feature updating within the proposed interactive total cost and construction duration feedback framework, and (2) conventional BIM model–level extraction that recomputes all aggregates from the entire BIM model at each authoring step. The t-dependent origin anchoring transformation is applied as a simple post-processing operation on the scalar model output, so its computational cost is negligible compared with feature extraction and base model inference. Latency analysis therefore focuses on mean inference duration per model call and on how feature extraction cost grows with project size.

Table 9 summarizes the mean per-call inference duration obtained from the 93 simulated projects for each base regressor and target, using the deployed ONNX models with t-dependent origin anchoring applied. Inference was measured on the ONNX runtime rather than in the original Python training environment, so that the reported numbers reflect the actual deployment setting. Across all configurations, mean inference time lies between 6.0417 and 6.0796 milliseconds, with standard deviations ranging from ±2.0321 to ±2.0336 milliseconds, corresponding to a spread of approximately 0.0379 milliseconds. Differences between regressors are therefore negligible in practice: all models respond in roughly six milliseconds per evaluation. This is well below one tenth of

Table 9
Mean inference duration for total cost and construction duration estimation models.

Base regressor	Target project performance	Inference duration (milliseconds) mean (standard deviation)
XGBoost	**Total cost**	6.0417 (±2.0334)
	Construction duration	6.0752 (±2.0322)
LightGBM	**Total cost**	6.0419 (±2.0335)
	Construction duration	6.0422 (±2.0336)
CatBoost	**Total cost**	6.0425 (±2.0334)
	Construction duration	**6.0796** (±2.0321)
HGBoost	**Total cost**	6.0452 (±2.0335)
	Construction duration	6.0457 (±2.0329)

Table 8
Tested accuracy for construction duration estimation under monotonicity constraints.

Base regressor	t-dependent origin anchoring	Monotonicity constraint	MAE (days) mean (standard deviation)	MAPE (%) mean (standard deviation)	MPE (%) mean (standard deviation)	R2 mean (standard deviation)
XGBoost	×	×	78.5 (±21.2)	25.9 (±11.3)	−7.0 (±13.8)	0.446 (±0.230)
		✓	87.4 (±16.0)	26.2 (±4.0)	−4.0 (±10.3)	0.433 (±0.265)
	✓	×	**78.9 (±19.1)**	**24.0 (±5.9)**	**−3.0 (±11.0)** **	**0.451 (±0.253)**
		✓	88.2 (±18.3)	25.4 (±3.6)	−1.5 (±9.4)	0.416 (±0.275)
LightGBM	×	×	78.0 (±20.5)	24.5 (±9.6)	−7.2 (±8.9)	0.540 (±0.194)
		✓	79.5 (±12.9)	24.1 (±6.5)	−6.6 (±7.5)	0.536 (±0.199)
	✓	×	**80.0 (±18.8)**	**23.7 (±5.4)**	**−3.5 (±5.5)** **	**0.531 (±0.189)**
		✓	81.4 (±14.8)	23.4 (±3.2)	−3.1 (±6.2)	0.524 (±0.207)
CatBoost	×	×	69.0 (±9.3)	22.6 (±9.3)	−5.7 (±13.4)	0.608 (±0.203)
		✓	210.0 (±21.4) ***	52.4 (±4.6) ***	51.3 (±4.6) ***	−1.537 (±0.384)
	✓	×	**70.0 (±6.3)**	**21.3 (±4.0)**	**−2.1 (±11.0)** **	**0.599 (±0.198)**
		✓	211.5 (±22.8) ***	52.8 (±5.1) ***	52.8 (±5.1) ***	−1.559 (±0.385)
HGBoost	×	×	81.4 (±13.7)	25.4 (±9.5)	−7.4 (±11.2)	0.514 (±0.165)
		✓	81.4 (±13.7)	25.4 (±9.5)	−7.4 (±11.2)	0.514 (±0.165)
	✓	×	**81.7 (±14.7)**	**23.4 (±5.2)**	**−3.1 (±8.5)** **	**0.517 (±0.175)**
		✓	81.7 (±14.8)	23.4 (±5.2)	−3.1 (±8.5)	0.518 (±0.175)

*✓ = monotonicity constraint / t-dependent origin anchoring transformation applied; × = not applied; ** = $p < 0.01$ and *** = $p < 0.001$ based on the Wilcoxon signed-rank test, using the case in which neither the monotonicity constraint nor the t-dependent origin anchoring transformation was applied as the baseline.

the 100-millisecond threshold typically associated with perceptually immediate feedback in direct manipulation interfaces, indicating that model inference itself does not limit interactive responsiveness.

To assess end-to-end latency from a user perspective, these inference times were combined with feature extraction times along the 93 mock authoring sequences. For the incremental, state-log-based mechanism, feature extraction recomputes only the contributions of the elements affected by the current event. Using the slowest deployed configuration (CatBoost for duration estimation) as a conservative case, total estimation time per authoring state remained consistently below the 100-millisecond real-time threshold for all simulated projects and all numbers of modeled elements considered in this study. In other words, under the proposed state-log-based incremental updating, even the slowest model configuration delivers feedback that is perceived as immediate throughout the entire as-authoring process.

In contrast, conventional BIM model–level extraction recomputes all relevant features from the full BIM model at every step by scanning all elements. In this scenario, end-to-end latency grows rapidly as the model becomes larger. On average across the 93 projects, total estimation time first exceeds the 100-millisecond real-time threshold when the BIM model contains only about 17 elements, at which point the conventional extraction path requires 103 milliseconds per estimation event (Fig. 11). This is 17 times slower than the corresponding state-log-based incremental update. As the number of elements increases, the gap widens further. When the model contains about 162 elements, total estimation time reaches roughly 1019 milliseconds, crossing the 1-s "cognitive operation" threshold commonly used in human–computer interaction. At this point, conventional extraction is about 162 times slower than the proposed interactive cost and duration feedback framework. For models with around 1763 elements, total estimation time rises to approximately 10,090 milliseconds, exceeding the 10-s "unit task" threshold and becoming clearly unsuitable for interactive use; in this regime, conventional extraction requires roughly 1667 times as much time as the state-log-based approach.

These results show that, while the choice of base regressor and the use of the t-dependent origin anchoring transformation have negligible impact on computational cost at inference time, the choice of feature extraction strategy is critical for real-time deployment. The proposed state-log-based incremental feature updating keeps total estimation latency within interactive bounds even as BIM models grow to include thousands of elements. In contrast, conventional full-model extraction quickly becomes a computational bottleneck as the number of elements and extracted fe atures increases, causing feedback delays that exceed established thresholds for responsive human–computer interaction.

6. Discussion

This section interprets the empirical results beyond the raw evaluation metrics, explains the contribution of the proposed method, and outlines current limitations and future work. Two questions guide the discussion: (1) Are the achieved accuracy sufficient for early-stage decision support? and (2) Why do estimated total cost and construduration behave so differently under the same modeling pipeline?

Table 10 summarizes the AACE cost estimate classification system and its expected accuracy ranges in relation to LOD in BIM-based projects. Because LOD is defined at the object level, its relation to estimate class depends not only on project stage but also on the estimation context. In general building design workflows, LOD 200 is commonly associated with schematic design and feasibility studies, while LOD 300 is more typical of design development and pre-tender budgeting. Higher LOD ranges may also appear in contractor- or fabricator-led estimating for bidding, procurement, or fabrication support.

The target LOD in this study is LOD 200, which corresponds to AACE Class 4. From this perspective, XGBoost and CatBoost without monotonicity constraints showed the best performance for cost estimation when the t-dependent origin anchoring transformation was applied. XGBoost achieved a test MAPE of 44.4% with an MPE of −14.3%, and CatBoost achieved a test MAPE of 41.2% with an MPE of −14.1%. These values lie within the Class 4 band, =suggesting that the proposed framework can already provide early design stage BIM users with cost feedback broadly comparable to traditional schematic design budgeting practices. For duration, the best model, CatBoost without monotonicity constraints and with t-dependent origin anchoring transformation, achieves a MAPE of 21.3% and an MPE of −2.1%. The relative error is

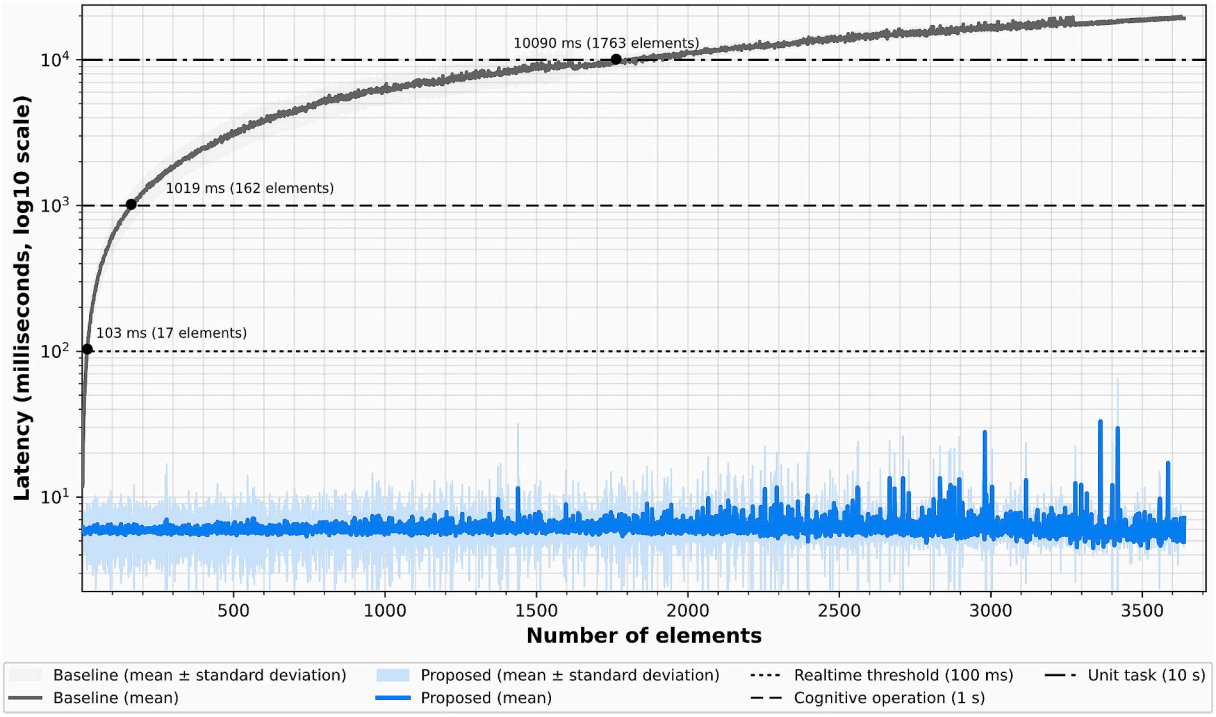


Fig. 11. Comparison of latency between state-log-based and BIM-model-based construction duration estimation (CatBoost).

Table 10
AACE international cost estimate classification (modified from [7]).

AACE Class	Level of development (LOD)	Project phase	Estimation context	Project definition	Expected range of accuracy (Low)	Expected range of accuracy (High)
Class 5	LOD 100	Strategic planning; concept screening	Designer-led early planning	0% to 2%	−50% to −20%	+30% to +100%
Class 4	LOD 200	Feasibility study; schematic design budgeting	Designer-led schematic design	1% to 15%	−30% to −15%	+20% to +50%
Class 3	LOD 300	Budgeting; pre-tender estimate	Designer-led design development	10% to 40%	−20% to −10%	+10% to +30%
Class 2	LOD 350	Bidding; project controls; change management	Contractor/fabricator-led construction-level estimating	30% to 75%	−15% to −5%	+5% to +20%
Class 1	LOD 400	Procurement; detailed project controls; detailed change management	Contractor/fabricator-led construction-level estimating	65% to 100%	−10% to −3%	+3% to +15%

smaller than the cost. If the same AACE style bands are applied to the duration estimation, these values fall close to the Class 3 band.

This suggests that the proposed framework can provide indicative early-stage feedback on project-level total cost and construction duration with practical relevance for TVD. Particularly for cost estimation, the achieved accuracy is notable because the predicted target is not limited to material costs or selected direct quantities, but represents the reported total project-level cost, including direct and indirect construction-related costs, administrative expenses, and profit. In addition, the dataset includes more than nine types of public projects in varying GFAs, indicating that the observed performance was achieved across heterogeneous project contexts rather than within a narrowly defined single building type. Likewise, the duration target represents the overall construction period from commencement to completion, rather than an activity-level schedule generated from detailed process information. Although the estimates are derived from BIM-log-based geometric quantities at LOD 200 without detailed material, construction-method, resource, or process information, they show meaningful potential to support TVD by indicating how ongoing early design changes may affect overall cost and duration targets.

Although both cost and duration estimators achieve accuracy levels what would typically be expected from LOD 200 information, their responses to shape-aware constraints differ. As observed in the empirical results, applying a linear t-dependent origin anchoring across the full $t(x)$ range led to performance degradation for duration estimation in some cases although it was not statistically significant, whereas cost estimation consistently benefited from the transformation. Also, the R^2 values range differently between cost (between 0.61 and 0.76) and duration (between 0.43 and 0.61) estimations. Such contrast suggests that the effectiveness of shape-aware constraints may depend on how well the imposed constraints align with the structural characteristics of the target variable.

Construction cost is closely linked to element-level quantities that accumulate in a largely continuous and additive manner as BIM authoring progresses. In this context, the t-dependent origin anchoring transformation effectively corrects boundary and near-boundary behavior while preserving the learned in-distribution relationships, thereby improving both accuracy and bias without distorting the underlying cost structure. Construction duration, by contrast, evolves in a more discrete and stepwise fashion. Project duration is often governed by higher-level organizational decisions such as floor-by-floor execution, construction phases, or trade sequencing that are only weakly reflected in schematic quantity aggregates.

As a result, globally imposing a linear anchoring behavior along the $t(x)$ axis can oversimplify the underlying dynamics of schedule progression and, in some cases, suppress meaningful local variations learned during the training. This suggests that duration estimation may require anchoring strategies or feature representations that better reflect actual construction behavior, rather than relying solely on a linear behavior constraint.

6.1. Contributions

Despite extensive research on BIM-based cost and schedule estimation and BIM log mining, providing early design stage total cost and construction duration estimation feedback remains largely disconnected from the on-going BIM authoring. Estimation is typically performed on interim or finalized design states, while BIM log mining has focused on post-hoc process analysis rather than supporting result-aware decision-making during design. As a result, designers receive cost and schedule feedback only after design actions have been taken, limiting their ability to align early-stage authoring decisions with project-level cost and schedule targets. This study contributes to the body of knowledge by addressing this disconnect.

First, this study provides a technical foundation to support early design stage TVD and advances BIM-based cost and schedule estimation by enabling continuous feedback during design authoring. Unlike conventional approaches that estimate performance only after exporting interim or final models, the proposed framework updates total cost and construction duration estimates incrementally in response to each authoring event with immediate interaction latency. By delivering estimated cost and duration as feedback at the time of action, the framework allows designers to remain continuously aware of project level implications as the design progresses, supporting more informed and target-aligned design decisions during early design stage authoring.

Second, this study introduces a conceptual reframing of BIM log mining as the enabling mechanism for such interactive estimation. Rather than considering BIM logs as records for post-hoc process analysis, the proposed framework uses enriched event-level state BIM logs as a live data substrate for estimation. By logging and incrementally aggregating geometric and semantic changes at each authoring event, BIM logs become an operational interface between BIM authoring and estimation. This reframing positions BIM log mining as an active decision support technique embedded within the authoring loop, rather than a retrospective analytic tool.

Third, this study proposes the t-dependent origin anchoring transformation to address a key deployment limitation of existing BIM-based cost and schedule estimation methods. Conventional machine learning regressors for BIM-based cost and schedule estimation generally require interim or finalized BIM models, as both training and inference typically rely on feature distributions derived from sufficiently developed design states. However, such settings do not provide the basis for as-designed estimation, because neither training data nor deployment inputs are typically available for the immediate estimation of cost and duration

consequences during ongoing design decisions. Using the element-level BIM log introduced in this study, the proposed post-estimation transformation enables logically consistent estimation from early BIM states while substantially reducing extrapolation violations. In doing so, it bridges the training–deployment gap by enabling regressors trained on finalized project states to remain usable during ongoing design, where cost and duartion implications must be inferred from early BIM states not represented in the training data.

6.2. Limitations and future directions

Several limitations of the current study point to concrete research directions. We did not conduct a dedicated comparison of alternative feature sets or systematically assess the individual and joint predictive contributions, interactions, and dependencies of the selected BIM features with respect to total cost and construction duration. The input feature set was limited to quantities and simple geometric aggregates drawn from prior BIM-based estimation studies and available for incremental updating during early design authoring. Further analysis is needed to identify which features and feature combinations contribute most strongly to predictive performance and to establish the conditions and ranges within which the resulting predictions remain reliable. Beyond these BIM-derived features, time-varying and project-specific factors, such as construction cost indices, regional market conditions, building function, procurement strategy, and contractor capability, are not modeled. The validation scope is further limited to schematic-design scenarios in which spatial configurations are represented through foundational BIM elements. Because relevant design information evolves from functional requirements and overall spatial organization in conceptual design to detailed architectural components in later stages, the current framework does not yet cover the full range of representations involved in architectural decision-making. Future work should integrate external data sources and project metadata, examine the predictive contributions and applicable ranges of different feature combinations for each performance target [53], and extend the framework to support broader design representations across stages.

Second, the reported performance is limited by the scope of the training dataset. The estimators were trained using BIM-derived element properties and reported project-level cost and duration outcomes from built public projects registered with the PPS of the Republic of Korea, all of which use reinforced concrete as their primary structural system. Although the dataset includes multiple public building types, the learned relationships may reflect the procurement conditions, cost-reporting conventions, and construction characteristics of Korean public projects. Therefore, their applicability to private developments, projects in other regional contexts, or buildings using substantially different structural systems remains to be validated. Furthermore, because the target outcomes are derived from built projects rather than from a large corpus of actual early-stage authoring trajectories, the proposed t-dependent origin anchoring transformation addresses logical reliability under intermediate-state inputs but does not eliminate the need for external validation using observed in-authoring design data.

Third, the current implementation computes the t-dependent progress index by normalizing each monotone quantity feature against a marginal lower reference and averaging the resulting values with equal weights. This assumes that all monotone quantitative features contribute equally and proportionally to authoring progress relative to the effective training support and that progress evolves similarly across estimation targets. In practice, these features may have different marginal distributions, scales, and levels of representation in the training data, while their relationships with cost and duration may vary across authoring stages, particularly in early BIM states. Uniform aggregation based only on marginal normalization may therefore not fully represent how heterogeneous features jointly characterize progress. Future work should investigate target- and feature-specific progress mappings, for example through learned feature weights or auxiliary progress models, to derive t-dependent origin anchoring transformations that more closely reflect empirical authoring dynamics.

Fourth, although the paper proposes a general logging schema and feature mapping procedure, the reference implementation of the proposed framework is tightly coupled to Autodesk Revit and its data structures. To support heterogeneous BIM authoring tools, the logging schema and feature mapping need to be further generalized. This includes defining tool agnostic element categories, parameter namespaces, and event types, and documenting how different platforms map their internal representations to this schema. Broader adoption of such a schema would enable cross tool analysis of authoring processes, more robust benchmarking of AI enhanced design assistance, and more personalized yet comparable analytics for different practices and authors.

Fifth, although the proposed framework demonstrated interactive latency and reasonable estimation accuracy, the current validation focuses primarily on algorithmic performance rather than real-world adoption in design practice. In particular, this study does not evaluate how designers interpret and use the provided feedback during actual BIM authoring nor examine how the framework fits into collaborative design decision-making workflows in practice. Therefore, the practical effectiveness of the framework for design decision-making should be interpreted with caution. Future work should include user studies and practical deployment evaluations to examine how well the proposed interactive feedback mechanism can be adopted in practice and how it can influence real design processes and outcomes.

Finally, the current work focuses on tree-based gradient boosting regressors and scalar project targets (i.e., total cost and construction duration). Future research can investigate how the same state log paradigm and shape constrained extrapolation ideas transfer to richer model classes such as neural networks or probabilistic models, to multi-output estimation such as joint cost and schedule distributions, and to downstream decision problems such as recommending design moves that improve the design to be aligned with the target. In all of these extensions, the core idea remains the same. BIM logs can be used not only to explain what has happened in past projects but also to provide feedback on what is likely to happen as a design evolves.

7. Conclusion

Early design decisions and subsequent authoring actions strongly influence total cost and construction duration. However, conventional estimation still relies on delayed expert feedback based on interim or final BIM models, or requires efforts for mapping BIM elements to predefined databases. This paper introduced the BIM-log-mining-based interactive total cost and construction duration feedback framework, which brings estimation results into the ongoing schematic design phase by combining BIM element state logging with shape-constrained machine learning estimation. The proposed framework includes three modules: (1) an event logging module, (2) a target value estimation module, and (3) a trends visualization module. The event logging module records element-level changes and maintains an up-to-date state log. The target value estimation module transforms this state log into aggregated features and applies tree-based gradient boosting regressors with a t-dependent origin anchoring transformation, thereby addressing low-count extrapolation violations that arise when models trained only on final-state project–performance pairs are queried on ongoing design states. The visualization module presents cost and duration trends directly in the authoring environment, enabling designers to see the consequences of design decisions as they are made.

The validation focused on three aspects: accuracy, low-count extrapolation behavior, and latency. The proposed t-dependent origin

anchoring transformation improved extrapolation behavior by removing origin bias and reducing stepwise monotonicity violations in early BIM states that were not represented in the training data. The framework also maintained reasonable estimation accuracy and immediate interactive responsiveness during ongoing authoring. Together, these results suggest that BIM-log-based cost and schedule estimation can provide stable and logically consistent feedback even for design states below the training distribution.

Beyond these findings from the validation, this study contributes to the field of BIM log mining by converting the focus toward an active design support mechanism. By operationalizing element-level state logs as a live interface between BIM authoring and cost and schedule estimation, the proposed framework enables interactive feedback to be generated continuously as schematic designs evolve. In this setting, BIM logs are no longer treated as records to be inspected after project completion, but as an integral part of the design loop that supports performance-aware decision-making at the time of action. While demonstrated here for total cost and construction duration, the same approach can be extended to other scalar performance indicators and enriched feature spaces as more detailed BIM states or external project data become available.

At the same time, the results expose clear limitations that point to future work. The current feature set is restricted to BIM-based quantities and simple geometry, where total cost is captured more reliably than construction duration and external drivers such as market conditions, construction methods, or site constraints remain unmodeled. The t-dependent progress index treats all monotone features as equally important and linear, even though their influence on performance can be heterogeneous and nonlinear. The reference implementation is limited to Autodesk Revit and to tree-based gradient boosting regressors. Addressing these limitations by learning more realistic progress measures, generalizing the logging schema across authoring platforms, and testing richer model families will be essential steps toward using BIM log mining not only to analyze completed projects but to steer design decisions under real project constraints.

By combining BIM element state logs with shape-constrained regressors, this study reframes BIM log mining from retrospective analysis to a mechanism for delivering interactive total cost and construction duration feedback during early design stage BIM authoring. The proposed framework demonstrates that, at the schematic design stage, it is possible to produce stable and logically consistent estimates with controlled extrapolation and real-time interactive latency. This allows designers to monitor how estimated outcomes change as they modify walls, rooms, and layouts, and to compare design alternatives directly within the authoring workflow rather than relying on experience or separate estimating tools. In doing so, the framework operationalizes TVD by enabling frequent and decision-synchronous updates of cost and schedule expectations during early stage design authoring.

CRediT authorship contribution statement

Suhyung Jang: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Project administration, Methodology, Investigation, Funding acquisition, Data curation, Conceptualization. **Ghang Lee:** Writing – review & editing, Investigation, Formal analysis, Conceptualization. **André Borrmann:** Writing – review & editing, Methodology, Conceptualization. **Seokho Hyun:** Writing – review & editing, Software, Investigation. **Junghun Lee:** Writing – review & editing, Software, Investigation. **Doyoon Park:** Writing – review & editing, Data curation, Conceptualization. **Seokheon Yun:** Writing – review & editing, Resources, Data curation.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work the authors used OpenAI's tool, ChatGPT in order to proofread the manuscript. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was supported by the National Research Foundation of Korea (NRF) grant funded by the Ministry of Science and ICT (Grant RS-2025-00522484 and RS-2025-23324132).

Appendix A. Supplementary data

Supplementary data to this article can be found online at https://doi.org/10.1016/j.autcon.2026.107184.

Data availability

The authors do not have permission to share data.

References

[1] F.K.T. Cheung, J. Rihan, J. Tah, D. Duce, E. Kurul, Early stage multi-level cost estimation for schematic BIM models, Autom. Constr. 27 (2012) 67–77, https://doi.org/10.1016/j.autcon.2012.05.008.
[2] T. Akanbi, J. Zhang, Design information extraction from construction specifications to support cost estimation, Autom. Constr. 131 (2021) 103835, https://doi.org/10.1016/j.autcon.2021.103835.
[3] P. MacLeamy, Collaboration, Integrated Information and the Project Lifecycle in Building Design, Construction and Operation. https://fr.scribd.com/document/474888363/, 2004 (accessed July 24, 2026).
[4] G. Ballard, Process benchmarks: target value design: current benchmark, Lean Constr. J. (2011) 79–84, https://doi.org/10.60164/71a1f5b4f.
[5] G. Ballard, P. Reiser, The St. Olaf college fieldhouse project: A case study in designing to target cost, in: 12th Annual Conference of the International Group for Lean Construction, Helsingor, Denmark, 2004, pp. 1–9. https://iglc.net/Papers/Details/325 (accessed July 24, 2026).
[6] S. Jang, G. Lee, M. Park, J. Lee, S. Suh, B. Koo, Semantic elaboration of low-LOD BIMs: inferring functional requirements using graph neural networks, Adv. Eng. Inform. 64 (2025) 103100, https://doi.org/10.1016/j.aei.2024.103100.
[7] P.R. Bredehoeft Jr., C. FAACE, L.R. Dysert, C. Life, T.W. Pickett, C. CEP, J. J. Borowicz, C. PSP, R.B. Brown, P.D.C. Donaldson, Cost Estimate Classification System-as Applied in Engineering, Procurement, and Construction for the Process Industries. https://aacei-pittsburgh.org/wp-content/uploads/2021/11/cost-estimating-classification-system.pdf, 2019.
[8] M.T. Hatamleh, M. Hiyassat, G.J. Sweis, R.J. Sweis, Factors affecting the accuracy of cost estimate: case of Jordan, Eng. Constr. Archit. Manag. 25 (2018) 113–131, https://doi.org/10.1108/ECAM-10-2016-0232.
[9] Z. Liu, J. Heer, The effects of interactive latency on exploratory visual analysis, IEEE Trans. Vis. Comput. Graph. 20 (2014) 2122–2131, https://doi.org/10.1109/TVCG.2014.2346452.
[10] S. Jang, G. Lee, S. Shin, H. Roh, Lexicon-based content analysis of BIM logs for diverse BIM log mining use cases, Adv. Eng. Inform. 57 (2023) 102079, https://doi.org/10.1016/j.aei.2023.102079.
[11] Autodesk, About Journal Files, Autodesk REVIT 2022 Help. https://help.autodesk.com/view/RVT/2022/ENU/?guid=GUID-477C6854-2724-4B5D-8B95-9657B636C48D, 2022 (accessed November 22, 2022).
[12] S. Yarmohammadi, D. Castro-Lacouture, Automated performance measurement for 3D building modeling decisions, Autom. Constr. 93 (2018) 91–111, https://doi.org/10.1016/j.autcon.2018.05.011.
[13] J. Abualdenien, A. Borrmann, A meta-model approach for formal specification and consistent management of multi-LOD building models, Adv. Eng. Inform. 40 (2019) 135–153, https://doi.org/10.1016/j.aei.2019.04.003.
[14] J. Abualdenien, A. Borrmann, Levels of detail, development, definition, and information need: a critical literature review, J. Inf. Technol. Constr. 27 (2022) 363–392, https://doi.org/10.36680/j.itcon.2022.018.
[15] S.-H. An, U.-Y. Park, K.-I. Kang, M.-Y. Cho, H.-H. Cho, Application of support vector machines in assessing conceptual cost estimates, J. Comput. Civ. Eng. 21 (2007) 259–264, https://doi.org/10.1061/(ASCE)0887-3801(2007)21:4(259).
[16] Z. Ma, Z. Wei, X. Zhang, Semi-automatic and specification-compliant cost estimation for tendering of building projects based on IFC data of design model, Autom. Constr. 30 (2013) 126–135, https://doi.org/10.1016/j.autcon.2012.11.020.

[17] E. Plebankiewicz, K. Zima, M. Skibniewski, Analysis of the first polish BIM-based cost estimation application, Procedia Eng. 123 (2015) 405–414, https://doi.org/10.1016/j.proeng.2015.10.064.
[18] A. Fazeli, M.S. Dashti, F. Jalaei, M. Khanzadi, An integrated BIM-based approach for cost estimation in construction projects, Eng. Constr. Archit. Manag. 28 (2020) 2828–2854, https://doi.org/10.1108/ECAM-01-2020-0027.
[19] H. Kim, K. Anderson, S. Lee, J. Hildreth, Generating construction schedules through automatic data extraction using open BIM (building information modeling) technology, Autom. Constr. 35 (2013) 285–295, https://doi.org/10.1016/j.autcon.2013.05.020.
[20] H. Moon, H. Kim, V.R. Kamat, L. Kang, BIM-based construction scheduling method using optimization theory for reducing activity overlaps, J. Comput. Civ. Eng. 29 (2015) 04014048, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000342.
[21] Z. Wang, E. Rezazadeh Azar, BIM-based draft schedule generation in reinforced concrete-framed buildings, Constr. Innov. 19 (2019) 280–294, https://doi.org/10.1108/CI-11-2018-0094.
[22] D. Park, S. Yun, Construction cost prediction using deep learning with BIM properties in the schematic design phase, Appl. Sci.-Basel 13 (2023) 7207, https://doi.org/10.3390/app13127207.
[23] H. Lee, S. Yun, Strategies for imputing missing values and removing outliers in the dataset for machine learning-based construction cost prediction, Build 14 (2024) 933, https://doi.org/10.3390/buildings14040933.
[24] Y. Zhang, H. Mo, Intelligent building construction cost optimization and prediction by integrating BIM and elman neural network, Heliyon 10 (2024) e37525, https://doi.org/10.1016/j.heliyon.2024.e37525.
[25] K. Sigalov, M. König, Recognition of process patterns for BIM-based construction schedules, Adv. Eng. Inform. 33 (2017) 456–472, https://doi.org/10.1016/j.aei.2016.12.003.
[26] C.I. Lagos, R.F. Herrera, A.F. Mac Cawley, L.F. Alarcón, Predicting construction schedule performance with last planner system and machine learning, Autom. Constr. 167 (2024) 105716, https://doi.org/10.1016/j.autcon.2024.105716.
[27] H. Wefki, M. Elnahla, E. Elbeltagi, BIM-based schedule generation and optimization using genetic algorithms, Autom. Constr. 164 (2024) 105476, https://doi.org/10.1016/j.autcon.2024.105476.
[28] D. Garcia, N. Furtado, Cost performance based design - using digital technology for cost performance simulation in the conceptual phase of design, in: Proceedings of the 33rd International Conference on Education and Research in Computer Aided Architectural Design in Europe (eCAADe), Vienna, Austria, 2015, p. 619, https://doi.org/10.52842/conf.ecaade.2015.1.619.
[29] Z. Pučko, N. Šuman, U. Klanšek, Building information modeling based time and cost planning in construction projects, Organ. Technol. Manag. Constr. 6 (2014) 958–971, https://doi.org/10.5592/otmcj.2014.1.6.
[30] S. Jang, G. Lee, J. Oh, J. Lee, B. Koo, Automated detailing of exterior walls using NADIA: natural-language-based architectural detailing through interaction with AI, Adv. Eng. Inform. 61 (2024) 102532, https://doi.org/10.1016/j.aei.2024.102532.
[31] S. Jang, G. Lee, BIM library transplant: bridging human expertise and artificial intelligence for customized design detailing, J. Comput. Civ. Eng. 38 (2024) 04024004, https://doi.org/10.1061/JCCEE5.CPENG-5680.
[32] S. Yarmohammadi, R. Pourabolghasem, D. Castro-Lacouture, Mining implicit 3D modeling patterns from unstructured temporal BIM log text data, Autom. Constr. 81 (2017) 17–24, https://doi.org/10.1016/j.autcon.2017.04.012.
[33] Y. Pan, L. Zhang, BIM log mining: exploring design productivity characteristics, Autom. Constr. 109 (2020) 102997, https://doi.org/10.1016/j.autcon.2019.102997.
[34] E. Forcael, A. Martinez-Rocamora, J. Sepulveda-Morales, R. Garcia-Alvarado, A. Nope-Bernal, F. Leighton, Behavior and performance of BIM users in a collaborative work environment, Appl. Sci.-Basel 10 (2020) 2199, https://doi.org/10.3390/app10062199.
[35] L. Zhang, B. Ashuri, BIM log mining: discovering social networks, Autom. Constr. 91 (2018) 31–43, https://doi.org/10.1016/j.autcon.2018.03.009.
[36] Y. Kim, G. Lee, I. Park, S. Chin, BIM-LogNet clustering for automated design and drawing productivity measurement, Adv. Eng. Inform. 71 (2026) 104279, https://doi.org/10.1016/j.aei.2025.104279.
[37] S. Kouhestani, M. Nik-Bakht, IFC-based process mining for design authoring, Autom. Constr. 112 (2020) 103069, https://doi.org/10.1016/j.autcon.2019.103069.
[38] Y. Pan, L. Zhang, Automated process discovery from event logs in BIM construction projects, Autom. Constr. 127 (2021) 103713, https://doi.org/10.1016/j.autcon.2021.103713.
[39] Y. Pan, L. Zhang, A BIM-data mining integrated digital twin framework for advanced project management, Autom. Constr. 124 (2021), https://doi.org/10.1016/j.autcon.2021.103564.
[40] Y. Pan, L. Zhang, BIM log mining: learning and predicting design commands, Autom. Constr. 112 (2020) 103107, https://doi.org/10.1016/j.autcon.2020.103107.
[41] C. Du, Z. Deng, S. Nousias, A. Borrmann, Predictive modeling: BIM command recommendation based on large-scale usage logs, Adv. Eng. Inform. 68 (2025) 103574, https://doi.org/10.1016/j.aei.2025.103574.
[42] A. Akbar, Y. Chang, J. Song, S. Lee, S. Park, J. Bae, S. Kwon, BIM log mining framework using deep learning for productivity assessment in construction facilities, J. Asian Archit. Build. Eng. 25 (2026) 3573–3588, https://doi.org/10.1080/13467581.2025.2508442.
[43] W. Gao, C. Wu, W. Huang, B. Lin, X. Su, A data structure for studying 3D modeling design behavior based on event logs, Autom. Constr. 132 (2021) 103967, https://doi.org/10.1016/j.autcon.2021.103967.
[44] S. Jang, S. Shin, G. Lee, Logging modeling events to enhance the reproducibility of a modeling process, in: Proceedings of the International Symposium on Automation and Robotics in Construction (ISARC) 2021, IAARC Publications, 2021, pp. 256–263, https://doi.org/10.22260/ISARC2021/0037.
[45] E.S. Muckley, J.E. Saal, B. Meredig, C.S. Roper, J.H. Martin, Interpretable models for extrapolation in scientific machine learning, Dig. Dis. 2 (2023) 1425–1435, https://doi.org/10.1039/D3DD00082F.
[46] S. Jang, G. Lee, Improving BIM Authoring Process Reproducibility with Enhanced BIM Logging, in: Proceedings of the 23rd International Conference on Construction Applications of Virtual Reality (CONVR) 2023, Firenze University Press, Florence, Italy, 2023, pp. 508–514, https://doi.org/10.36253/979-12-215-0289-3.49.
[47] G. Lee, J. Won, S. Ham, Y. Shin, Metrics for quantifying the similarities and differences between IFC Files, J. Comput. Civ. Eng. 25 (2011) 172–181, https://doi.org/10.1061/(ASCE)CP.1943-5487.0000077.
[48] L. Prokhorenkova, G. Gusev, A. Vorobev, A.V. Dorogush, A. Gulin, CatBoost: unbiased boosting with categorical features, in: Proceedings of the 32nd International Conference on Neural Information Processing Systems, Curran Associates Inc., 2018, pp. 6639–6649, https://doi.org/10.5555/3327757.3327770.
[49] T. Chen, C. Guestrin, XGBoost: a scalable tree boosting system, in: Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, ACM, San Francisco California USA, 2016, pp. 785–794, https://doi.org/10.1145/2939672.2939785.
[50] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, T.-Y. Liu, Lightgbm: a highly efficient gradient boosting decision tree, in: Proceedings of the 31st International Conference on Neural Information Processing Systems (NeurIPS), Long Beach, California, USA, 2017, pp. 3149–3157, https://doi.org/10.5555/3294996.3295074.
[51] A. Guryanov, Histogram-based algorithm for building gradient boosting ensembles of piecewise linear decision trees, in: Analysis of Images, Social Networks and Texts: 8th International Conference, AIST 2019, Kazan, Russia, July 17–19, 2019, Revised Selected Papers, Springer-Verlag, Berlin, Heidelberg, 2019, pp. 39–50, https://doi.org/10.1007/978-3-030-37334-4_4.
[52] A. Habboush, B. Elzaghmouri, O.A. Altiti, Integrating building information (BIM) and artificial intelligence to enhance cost and schedule planning in energy-conscious infrastructure projects, Asian, J. Civ. Eng. 27 (2026) 2227–2241, https://doi.org/10.1007/s42107-025-01612-4.
[53] X. Chen, M.M. Singh, P. Geyer, Utilizing domain knowledge: robust machine learning for building energy performance prediction with small, inconsistent datasets, Knowl.-Based Syst. 294 (2024) 111774, https://doi.org/10.1016/j.knosys.2024.111774.